A POPULATION GENETICS SELF-STUDY PROJECT

# Why a beneficial allele can disappear

Build, run, and understand a Moran model in Julia

A guide from first principles

One small model. Calculations by hand. Readable code.
Repeated experiments. An exact mathematical answer.

Start with Chapter 1. You do not need to know population genetics, probability notation, or Julia. Every chapter builds on the earlier chapters. The supplied files let you run the same calculations yourself.

Prepared 27 October 2026. This is a teaching project using an established model; it is not a claim of a new biological discovery.

# How to use this guide

The aim is understanding. You should eventually be able to explain what the model assumes, why its equations have the form they do, what each part of the code calculates, and how to tell whether the results are plausible.

Do not try to memorise all the equations. At first, concentrate on the five-individual example. Recalculate it with a pen. Then run the demonstration. When the program prints a number, ask which biological event or mathematical quantity that number represents.

1. **First sitting:** read Chapters 1–3 and work through one event on paper.
2. **Second sitting:** read Chapters 4–5 and run the small demonstration. Change only one input at a time.
3. **Third sitting:** read Chapter 6 and understand why repeated runs differ.
4. **Fourth sitting:** work slowly through Chapter 7. Derive the answer on paper before looking at the code again.
5. **Fifth sitting:** read Chapters 8–9 and learn to interpret the figures.
6. **Sixth sitting:** use Chapter 10 to rerun the complete study and explain the project aloud without reading the text.

These are learning stages, not deadlines. It is reasonable to spread them over several weeks. The short exercises include answers so that you can check yourself immediately. Try covering the answer before reading it.

**What you receive.** The project contains the editable Julia code, automated checks, generated data, editable LaTeX source, and this PDF. It contains no real genetic dataset. Every simulation result describes the stated mathematical model.

**What "from the beginning" means here.** We introduce the biology, probability, algebra, calculus, and programming needed for this particular model. This is not a complete course in genetics or Julia. When a more advanced tool is used for checking, we explain what it checks and work through a smaller version by hand.

**Reading code in the PDF.** A grey block contains code. Numbers at its left identify lines; they are not part of the program. Some long lines wrap across the page. An explanation below the block connects its instructions to the model. The complete original files are included in the appendices.

# Contents

**How to use this guide** **i**

**1 The question, the biology, and the symbols** **1**

1.1 What are we trying to understand? . . . . . 1

1.2 The biological words, one at a time . . . . . 1

1.3 Describe the population with a few symbols . . . . . 2

1.4 Parameters and the changing state . . . . . 4

1.5 The assumptions we agree to use . . . . . 4

1.6 What one event means for time . . . . . 4

1.7 How to use this guide . . . . . 5

**2 The probability ideas we need** **6**

2.1 Probability describes a chance . . . . . 6

2.2 Fractions and equal chances . . . . . 6

2.3 Opposite events and the total probability . . . . . 7

2.4 Conditional probability: state what is already known . . . . . 7

2.5 Why we multiply probabilities . . . . . 7

2.6 Why we add probabilities . . . . . 8

2.7 A random variable is a numerical outcome . . . . . 8

2.8 Expected value: a probability-weighted average . . . . . 9

2.9 From probability to repeated computer experiments . . . . . 9

2.10 The three questions to ask before a calculation . . . . . 10

**3 Build one Moran event from its biological rules** **11**

3.1 Start with a known population state . . . . . 11

3.2 Choose the parent using reproductive weights . . . . . 11

3.3 Choose the individual for replacement uniformly . . . . . 12

3.4 List all four parent–replacement combinations . . . . . 12

3.5 Name the three transition probabilities . . . 13
3.6 Work through the advantageous example completely . . . 13
3.7 Repeat the calculation when selection is absent . . . 14
3.8 Why we keep unchanged events . . . 15

**4 The Julia you need, starting from the beginning 16**

4.1 What is a program? . . . 16
4.2 Two places where you can type commands . . . 16
4.3 Names store values . . . 17
4.4 Comments explain; they do not calculate . . . 17
4.5 Functions are reusable instructions . . . 17
4.6 Ask questions with comparisons . . . 18
4.7 Loops repeat instructions . . . 19
4.8 Arrays store several values in order . . . 19
4.9 Random numbers select events . . . 19
4.10 More than one answer from a function . . . 20
4.11 Files, libraries, and the project . . . 20

**5 Read the actual model code, one step at a time 21**

5.1 The map before the details . . . 21
5.2 Function 1: reject inputs that do not describe our model . . . 22
5.3 Function 2: calculate the three event probabilities . . . 22
5.4 Function 3: use a random number to choose the next count . . . 24
5.5 Function 4: record an entire population history . . . 24
5.6 Function 5: run until loss or fixation . . . 26
5.7 Function 6: calculate the exact answer accurately . . . 27
5.8 Function 7: check the exact formula by another method . . . 28
5.9 Function 8: calculate uncertainty from repeated experiments . . . 29
5.10 Function 9: calculate a deterministic comparison . . . 30

**6 Learning from repeated experiments 32**

6.1 One history cannot tell us a probability . . . 32
6.2 What the repetition loop does . . . 33
6.3 Random seeds: repeatable does not mean identical trials . . . 33
6.4 Why estimates change from one batch to another . . . 34
6.5 A rare outcome needs enough repetitions . . . 35

6.6 Confidence intervals: show a range, not just a point . . . 35
6.7 How this applies to our full study . . . 36

**7 Derive the exact probability of fixation 38**
7.1 An independent answer for checking the simulations . . . 38
7.2 Start with the two answers we already know . . . 38
7.3 Why the present count is enough . . . 38
7.4 Condition on the first event . . . 39
7.5 Rearrange the recurrence without skipping steps . . . 39
7.6 Name the differences between neighbouring answers . . . 40
7.7 Follow the pattern of differences . . . 40
7.8 Calculate the geometric sum explicitly . . . 41
7.9 Handle neutrality separately . . . 42
7.10 A complete five-individual calculation . . . 42
7.11 Check the answer using simultaneous equations . . . 43
7.12 A preview of stable numerical evaluation . . . 43
7.13 What we have established . . . 44

**8 What an average-change model can and cannot predict 45**
8.1 Two useful questions, two different calculations . . . 45
8.2 First calculate the expected change in the allele count . . . 45
8.3 Convert a count change into a frequency change . . . 46
8.4 Turn that expected change into one deterministic path . . . 46
8.5 Why this is not usually the exact average of the stochastic runs . . . 47
8.6 Time units and the differential equation . . . 48
8.7 An advantageous allele can have better odds and still usually disappear . . . 48
8.8 Build the weak-selection approximation step by step . . . 49
8.9 Understand the small and large parameter limits . . . 50
8.10 Measure approximation error rather than guessing . . . 50
8.11 What the product $Ns$ does and does not mean . . . 50
8.12 One initial copy is not the same as one initial frequency . . . 51

**9 Read the results, one figure at a time 52**
9.1 What was actually run? . . . 52
9.2 Figure 1: how can a growing deterministic curve coexist with loss? . . . 52
9.3 Figure 2: do the simulations agree with the exact calculation? . . . 55

9.4 Do larger populations give a new single copy a better chance? . . . 58
9.5 Figure 3: how wrong is a useful approximation? . . . 58
9.6 What have we learned, and what have we not learned? . . . 59

**10 Run, inspect, and reproduce the project 60**

10.1 Three things that are easy to confuse . . . 60
10.2 Open the project folder in a terminal . . . 60
10.3 Run the small demonstration first . . . 61
10.4 Read every part of the demonstration script . . . 61
10.5 Run the automated checks . . . 63
10.6 Choose a quick study or the full study . . . 64
10.7 Read the study script: loading and writing a CSV row . . . 64
10.8 Read the study script: the fixation grid . . . 65
10.9 Read the study script: compare initial conditions . . . 67
10.10 Read the study script: check the approximation . . . 67
10.11 Read the study script: save example trajectories . . . 68
10.12 Read the study script: record the run information . . . 70
10.13 Read the study script: the main function . . . 71
10.14 Turn full results into the document inputs . . . 72
10.15 Compile or open the LaTeX guide . . . 72
10.16 Make a small change and understand what it changes . . . 72
10.17 Diagnose a message before drawing a conclusion . . . 73

**A A reference sheet and a teach-back check 74**

A.1 The main biological words . . . 74
A.2 Read the mathematical notation in ordinary words . . . 75
A.3 Programming terms in plain language . . . 76
A.4 Connect mathematical ideas to the code . . . 77
A.5 Five distinctions to keep clear . . . 78
A.6 Teach the model back without looking . . . 78
A.7 A final practice exercise with a full answer . . . 80

**B The complete runnable source 81**

B.1 The model: src/MoranModel.jl . . . 81
B.2 The demonstration: scripts/demo.jl . . . 84
B.3 The full study script . . . 84

B.4 The automated checks: test/runtests.jl . . . . . 88
B.5 Preparing the guide's plot inputs . . . . . 89

**References** **92**

# Chapter 1

# The question, the biology, and the symbols

## 1.1 What are we trying to understand?

Imagine a small population. Most individuals carry one genetic type. A few individuals carry another type. The second type gives its carriers a reproductive advantage. Does that advantage guarantee that the second type will spread through the whole population?

No. An advantage changes the chances of reproduction. It does not determine the outcome of each reproduction event. A rare type can disappear before its advantage has much opportunity to act.

Our question is therefore precise: *what is the probability that an initially rare, advantageous type eventually occupies the whole population?* We will answer this question in two ways. First, we will repeat a random computer experiment. Second, we will calculate an exact mathematical answer for the model. Comparing the two answers will help us check our code.

The model is deliberately small. Its purpose is to make the reasoning visible. It is not a claim that every biological population follows these rules.

## 1.2 The biological words, one at a time

### 1.2.1 Population, individual, and genetic type

A **population** is the collection of individuals included in our model. An **individual** is one member of that collection. We follow one genetic difference. An **allele** is a version of a genetic locus, meaning a particular location or inherited genetic feature under consideration.

We call the two versions type A and type B. These letters are labels. We do not need to name a real gene. Type A is the *focal* type: it is the type whose fate we follow. Type B is the *resident* type: it is the alternative already present in the population.

We use a **haploid** model. Each individual carries one copy of the locus we follow, so each individual is either A or B. We do not have an AB genotype in this model. We do not model diploid inheritance, dominance, or mating. This is a modelling choice, not a description of all organisms.

Sometimes we call A the *mutant* type. Here that word describes the type whose initial arrival we imagine. It does not mean new mutations continue to appear during a run. After setting the initial state, we allow no further mutation.

### 1.2.2 Loss and fixation

Type A is **lost** when no A individuals remain. It has reached **fixation** when every individual is A.

These outcomes concern this modelled population and this locus. Fixation does not mean that all organisms everywhere carry A. Loss of A does not mean that the whole population has died. The population still exists; it now contains only B individuals.

Because we allow no new mutation, A cannot return after it has been lost. Likewise, B cannot return after A has fixed. These are **absorbing states**: once the process enters one, it stays there.

### 1.2.3 Selection and drift

**Selection** means that the genetic types differ in their expected reproductive success. In this project, A can have a greater chance of being chosen as a parent than B.

**Genetic drift** is random change in genetic composition caused by finite random sampling. Even if the two types have equal reproductive weights, the number of A individuals can move up or down by chance.

Selection and drift operate in the same run. We do not turn randomness off when we give A an advantage. Instead, we change the probabilities used by the random experiment.

> Suppose there are five individuals, and only one is A. A has a reproductive advantage. Nevertheless, a B individual might be selected as the next parent, and the one A individual might be selected for replacement. If that happens, the population becomes all B. The advantage did not disappear. The A lineage disappeared before reproducing successfully.

## 1.3 Describe the population with a few symbols

Mathematical symbols save us from writing long sentences repeatedly. Every symbol below has a concrete meaning. A symbol is not a new biological object.

### 1.3.1 The population size

The capital letter $N$ means the total number of individuals. We assume $N \geq 2$. The sign $\geq$ means "greater than or equal to". Thus our population contains at least two individuals.

We keep $N$ fixed during a simulation. For example, $N = 5$ means the population always contains five individuals.

### 1.3.2 The number of A individuals

The lowercase letter $i$ means the current number of A individuals. It can take the whole-number values

$$i = 0, 1, 2, \ldots, N.$$

The dots mean that we continue the same counting pattern. The number of B individuals is $N - i$: the total minus the number that are A.

For $N = 5$ and $i = 2$, there are two A individuals and three B individuals. For $i = 0$, A has been lost. For $i = N$, A has fixed.

The complete state of this model is the count $i$. We do not need the positions, ages, names, or family histories of the individuals. Our rules will depend on the current count and the fixed parameters only.

### 1.3.3 The frequency of A

The letter $x$ means the fraction of the population that is A. This is called the **allele frequency**. We calculate it by dividing the A count by the population size:

$$x = \frac{i}{N}.$$

The fraction bar means division. A frequency of 0 means no individuals are A. A frequency of 1 means all individuals are A.

For example, two A individuals in a population of five give

$$x = \frac{2}{5} = 0.4 = 40\%.$$

The count and frequency describe the same population in different units. The count is two individuals. The frequency is forty percent.

### 1.3.4 Reproductive weight and selective advantage

We give each B individual a reproductive weight of 1. The letter $s$ means the relative reproductive advantage assigned to A. In our main study, $s \geq 0$. Each A individual has reproductive weight $1 + s$.

We also use the letter $r$ for that same A weight:

$$r = 1 + s.$$

The two symbols are useful for different purposes. The symbol $s$ makes the size of the advantage easy to describe. The symbol $r$ makes some algebra shorter.

If $s = 0.5$, then $r = 1 + 0.5 = 1.5$. One A individual has one and a half times the reproductive weight of one B individual. We can also say that its weight is fifty percent larger.
This does *not* mean that A has a fifty percent probability of fixation. It does not even mean that the next parent has a fifty percent chance of being A. The chance of choosing an A parent also depends on how many A and B individuals are present.

When $s = 0$, we have $r = 1$. Both types have equal reproductive weight. We call this the **neutral** case. Genetic drift still occurs.

## 1.4 Parameters and the changing state

A **parameter** is a setting chosen for a run. Here $N$ and $s$ are parameters. Once the run starts, their values stay fixed. The value $r = 1 + s$ is fixed too.

The **state** is the current count $i$. That count changes as the run proceeds. The frequency $x = i/N$ changes with it.

We must also choose an initial count. For the main experiment, we start with one A individual. This is a choice about the starting state, not a claim that one A individual remains throughout the run.

| Symbol | Meaning | Example |
|---|---|---|
| $N$ | Total number of individuals | 5 |
| $i$ | Current number of A individuals | 2 |
| $N - i$ | Current number of B individuals | 3 |
| $x$ | Current A frequency, $i/N$ | 0.4 |
| $s$ | Relative reproductive advantage | 0.5 |
| $r$ | A reproductive weight, $1 + s$ | 1.5 |

## 1.5 The assumptions we agree to use

Our population is **well mixed**. Every individual can be selected for replacement. We do not assign individuals to different locations or groups. There is no migration into or out of the population.

At each model event, one parent produces one offspring. That offspring has the same type as its parent. One individual is selected for replacement, and the offspring takes that individual's place. There is no net change in population size. The selected individual can be the parent itself. We will explain why that is allowed when we derive the event probabilities.

The reproductive weights are constant. A does not become more or less advantageous as the environment changes, because this model has no changing environment. Its advantage also does not depend on its frequency.

These assumptions allow an exact answer. They also limit what the answer can tell us about a real population. A careful report states both points.

## 1.6 What one event means for time

One **event** is one attempt at reproduction and replacement. Some events increase the A count. Some decrease it. Some leave it unchanged.

We count unchanged events too. They are still reproduction and replacement attempts. Omitting them would change the meaning of elapsed time.

For plotting, we may divide the number of events by $N$. We call this time in *generations*. It is a conventional time unit: one generation means $N$ event attempts. It does not mean that each individual reproduced exactly once, or that all individuals were replaced simultaneously.

**Questions.** A run has $N = 10$, starts with $i = 3$, and uses $s = 0.2$. How many B individuals are present? What is the A frequency? What is the A reproductive weight? What do the two absorbing states mean? If twenty events have occurred, how many model generations have passed?
**Worked answers.** There are $N - i = 10 - 3 = 7$ B individuals. The A frequency is $x = i/N = 3/10 = 0.3$, or thirty percent. The A weight is $r = 1 + s = 1 + 0.2 = 1.2$. At $i = 0$, A is lost and all ten individuals are B. At $i = 10$, A is fixed and all ten are A. Twenty events divided by ten individuals gives two model generations.

## 1.7 How to use this guide

Read one worked example before moving to its general equation. Write down the numbers yourself. When a new symbol appears, say its meaning aloud. For example, read $i/N$ as "the number of A individuals divided by the total number of individuals".

When we reach the code, distinguish three questions: what biological event does this instruction represent, what mathematical quantity does it compute, and what does Julia do when it executes the instruction? Understanding all three connections is more useful than memorising the code.

You do not need to understand every later chapter immediately. At this point, the essential idea is simple: we will follow a count, change it using explicit random rules, and ask which boundary it eventually reaches.

# Chapter 2

# The probability ideas we need

## 2.1 Probability describes a chance

A **probability** is a number between zero and one. Zero means that an outcome is impossible under the stated rules. One means that it is certain. A probability of 0.25 is the same as one quarter or twenty-five percent.

An **event**, in probability language, is a specified outcome or set of outcomes. For example, "the selected individual is A" is an event. The word event will also refer to one reproduction and replacement attempt in the Moran model. The surrounding sentence will tell us which meaning is used.

We write $\Pr(A)$ for the probability of a probability event called $A$. Here $A$ is a temporary name for an event, not necessarily the genetic type A. For a concrete example, we can avoid a letter and write Pr(selected individual is A).

Probabilities concern the rules and information available before a draw. Once a particular draw has happened, its outcome is known. The fact that an unlikely outcome occurred does not mean its original probability was wrong.

## 2.2 Fractions and equal chances

Suppose a population contains five individuals. Two are A and three are B. If each individual is equally likely to be selected, there are five equally likely possibilities. Two of them meet the description "selected individual is A". Therefore

$$\Pr(\text{selected individual is A}) = \frac{2}{5} = 0.4.$$

The rule "favourable possibilities divided by all possibilities" works here because the individual possibilities have equal chances. We must not use it without checking that condition. Later, choosing a parent will involve unequal reproductive weights. Choosing an individual for replacement will still give equal chances to all individuals.

Think of five equally sized tickets in a bag. Two tickets say A and three say B. Drawing one ticket uniformly means that each ticket has probability 1/5. The total probability of drawing A is $1/5 + 1/5 = 2/5$. We add the probabilities because the two A tickets are

different possible outcomes of the same draw.

## 2.3 Opposite events and the total probability

If there are only A and B individuals, a selected individual must be one or the other. It cannot be neither. It also cannot be both in our haploid model. Their probabilities therefore add to one:

$$\Pr(\text{A selected}) + \Pr(\text{B selected}) = 1.$$

If the first probability is 0.4, the second must be $1 - 0.4 = 0.6$.

The event "not A" is called the **complement** of "A". In general, the probability of a complement is one minus the probability of the original event. We will use this fact when calculating the chance that the allele count stays unchanged.

## 2.4 Conditional probability: state what is already known

The phrase **conditional probability** means a probability calculated given some specified information. A vertical bar is read as "given". For example,

$$\Pr(\text{A selected} \mid i = 2, N = 5)$$

means "the probability that A is selected, given that there are two A individuals in a population of five". The symbols $i$ and $N$ have the meanings introduced in Chapter 1.

The information matters. If there were no A individuals, the chance would be zero. If all five individuals were A, the chance would be one.

For a more abstract statement, let $A$ and $B$ name two probability events. The expression $\Pr(B \mid A)$ is the probability that $B$ occurs when we know that $A$ has occurred. If $\Pr(A) > 0$, its definition is

$$\Pr(B \mid A) = \frac{\Pr(A \text{ and } B)}{\Pr(A)}.$$

The numerator is the probability that both events occur. Dividing by $\Pr(A)$ restricts our attention to occasions on which $A$ occurred.

We will not need to manipulate this definition repeatedly. The important habit is to ask: *what information is this probability conditioned on?*

## 2.5 Why we multiply probabilities

Rearranging the definition above gives the general multiplication rule:

$$\Pr(A \text{ and } B) = \Pr(A)\Pr(B \mid A).$$

The expression on the right means the chance of the first event, multiplied by the chance of the second event given the first.

Two events are **independent** when knowing that the first occurred does not change the probability of the second. In that case $\Pr(B \mid A) = \Pr(B)$, so the multiplication rule becomes

$$\Pr(A \text{ and } B) = \Pr(A)\Pr(B).$$

Suppose the chance of choosing an A parent is 0.5. Suppose an independent replacement draw has probability 0.6 of selecting a B individual. The chance of choosing an A parent *and* replacing a B individual is

$$0.5 \times 0.6 = 0.3.$$

As an intuitive count, imagine one hundred repetitions. About fifty choose an A parent. Of those fifty, about sixty percent, or thirty, replace B. Thirty out of one hundred is 0.3. These are expected counts, not promises about a particular batch of one hundred repetitions.

In our model, the parent and replacement draws are independent *given the current population state*. They both use the same current counts. We do not remove the parent before choosing a replacement. We also do not rule out selecting the same individual twice.

This conditional statement is precise. If we mixed observations from many different population states without recording those states, the apparent relationship between the draws could be different.

## 2.6 Why we add probabilities

Events are **mutually exclusive** when they cannot both happen in the same trial. The outcomes "the A count goes up", "the A count goes down", and "the A count stays the same" are mutually exclusive.

When events are mutually exclusive, we find the probability that one or another occurs by adding their probabilities. For example, suppose an event has probability 0.3 of increasing the count and probability 0.2 of decreasing it. Then

$$\Pr(\text{count changes}) = 0.3 + 0.2 = 0.5.$$

The remaining probability is $1 - 0.5 = 0.5$, so the count stays unchanged with probability 0.5.

Do not add probabilities in this simple way when the events overlap. If the same possible outcome belongs to both events, adding would count it twice. The three count outcomes above do not overlap.

## 2.7 A random variable is a numerical outcome

A **random variable** is a numerical quantity determined by a random outcome. Let $Y$ denote the change in the A count during one event. The possible values of $Y$ are

$$Y = 1, \qquad Y = -1, \qquad \text{or} \qquad Y = 0.$$

The value 1 means one more A individual. The value $-1$ means one fewer A individual. The value 0 means no change.

The symbol names the quantity before we know which value the random event will produce. A realised value is the actual value produced in one trial.

## 2.8 Expected value: a probability-weighted average

The **expected value** of a random variable is its average when each possible value is weighted by its probability. We write $\mathbb{E}[Y]$ for the expected value of $Y$. The stylised letter $\mathbb{E}$ is read as "expectation".

Suppose $Y$ equals 1 with probability 0.3, equals $-1$ with probability 0.2, and equals 0 with probability 0.5. Multiply each value by its probability and add:

$$\begin{aligned}\mathbb{E}[Y] &= 1(0.3) + (-1)(0.2) + 0(0.5) \\ &= 0.3 - 0.2 + 0 \\ &= 0.1.\end{aligned}$$

The expected change is one tenth of an individual per event. An actual event cannot add one tenth of an individual. It adds one, removes one, or changes nothing. An average does not have to be a possible individual outcome.

For intuition, imagine ten events that each begin from the *same* population state. On average across many such batches, three events increase the count and two decrease it. The average total change is one, so the average change per event is $1/10 = 0.1$.

This thought experiment resets the starting state before each event. In a single evolving population, a changed count changes the probabilities of later events. We cannot assume that ten successive events in one trajectory all use the original probabilities.

### 2.8.1 Expected change in frequency

The frequency is the count divided by the fixed population size $N$. Let $\Delta x$ mean the change in frequency during one event. The Greek capital letter $\Delta$ is read as "change in". Since the count change is $Y$, the frequency change is $\Delta x = Y/N$.

For the example above with $N = 5$, the three possible frequency changes are $1/5$, $-1/5$, and zero. Their expected value is

$$\mathbb{E}[\Delta x] = \frac{0.1}{5} = 0.02.$$

Again, a positive average does not rule out a decrease in the next event.

## 2.9 From probability to repeated computer experiments

Suppose we repeat a complete simulation many times, always starting with the same parameters and initial count. One complete run is a **replicate**. Each replicate ends with A either lost or fixed.

Let $R$ denote the number of replicates. Let $K$ denote the number of those replicates in which A fixes. A natural estimate of the fixation probability is

$$\widehat{p} = \frac{K}{R}.$$

The symbol $\widehat{p}$, read as "p hat", means an estimated probability. The hat reminds us that this number is computed from a finite sample of random outcomes. We use the letter $p$, without a hat, for the underlying probability specified by the model.

If twelve out of one hundred runs fix, then $\widehat{p} = 12/100 = 0.12$. Another batch might give nine or fourteen fixations. The rules can stay the same while the observed count differs.

Larger numbers of replicates usually give more precise estimates. They do not change the underlying model probability. A later chapter will put a confidence interval around the estimate and explain what that interval means.

A **random seed** is an initial setting for the computer's random-number generator. Recording a seed helps reproduce a particular computation. It does not turn one random outcome into a universal result.

**Question 1.** A parent draw selects A with probability 0.4. An independent replacement draw selects B with probability 0.6. What is the probability of both outcomes?
**Answer.** Multiply the probabilities: $0.4 \times 0.6 = 0.24$. We multiply because both outcomes are required, and the two draws are independent given this state.
**Question 2.** The count increases with probability 0.24 and decreases with probability 0.24. What is the probability of no change? What is the expected count change?
**Answer.** The probability of a change is $0.24 + 0.24 = 0.48$. The probability of no change is $1 - 0.48 = 0.52$. The expected change is $1(0.24) + (-1)(0.24) + 0(0.52) = 0$. An expected change of zero does not mean that every event leaves the count unchanged.
**Question 3.** A set of $R = 200$ complete runs contains $K = 30$ fixations. What probability do we estimate?
**Answer.** The estimate is $\widehat{p} = K/R = 30/200 = 0.15$. Fifteen percent of this particular batch fixed. The true model probability need not be exactly 0.15.

## 2.10 The three questions to ask before a calculation

Before multiplying, ask whether you need *both* events and whether the second probability is conditional on the first. Before adding, ask whether the alternatives overlap. Before interpreting an average, ask whether it is a statement about many repetitions or about one actual trajectory.

Those checks will carry most of the probability reasoning in this project.

# Chapter 3

# Build one Moran event from its biological rules

## 3.1 Start with a known population state

We now turn the biological description into explicit probabilities. The population size is $N$. The current A count is $i$, so the current B count is $N - i$. Each A individual has reproductive weight $r = 1 + s$, and each B individual has weight 1.

One event has two selections. First we choose a parent. Then we independently choose an individual for replacement from the *current* population. The parent's offspring has its parent's type and replaces the selected individual. We use the old counts for both selections.

We are not choosing two distinct individuals without replacement. The parent can also be selected for replacement. In that case its offspring takes its place and the type count does not change. Allowing this possibility is part of the particular Moran model used here.

## 3.2 Choose the parent using reproductive weights

### 3.2.1 Add the weights within each type

There are $i$ A individuals, each with weight $r$. Their total weight is $ri$, which means $r$ multiplied by $i$. There are $N - i$ B individuals, each with weight one. Their total weight is $N - i$.

Let $W_i$ denote the total reproductive weight of the population when the A count is $i$. The subscript $i$ reminds us that the total depends on the current state. Adding the two type weights gives

$$W_i = ri + N - i.$$

### 3.2.2 Divide a type's weight by the total

Choosing a parent proportional to weight means that a type's chance is its total weight divided by the population's total weight:

$$\Pr(\text{parent is A} \mid i) = \frac{ri}{W_i},$$
$$\Pr(\text{parent is B} \mid i) = \frac{N-i}{W_i}.$$

The notation $\mid i$ means that these chances are calculated given the current count $i$; the fixed values of $N$ and $r$ are understood. The numerators add to $W_i$, so the probabilities add to one.

Take $N = 5$, $i = 2$, and $s = 0.5$. Then $r = 1.5$. The two A individuals have total weight $1.5 \times 2 = 3$. The three B individuals also have total weight $1 \times 3 = 3$. The total is $W_i = 3 + 3 = 6$.
The chance of choosing an A parent is $3/6 = 0.5$. The chance of choosing a B parent is also $3/6 = 0.5$. Although there are fewer A individuals, their greater per-individual weight makes the two types equally likely to supply the next parent in this particular state.

## 3.3 Choose the individual for replacement uniformly

The replacement selection does not use reproductive weights. Every current individual has probability $1/N$ of being selected. Because there are $i$ A individuals and $N - i$ B individuals,

$$\Pr(\text{replacement is A} \mid i) = \frac{i}{N},$$
$$\Pr(\text{replacement is B} \mid i) = \frac{N-i}{N}.$$

The phrase "replacement is A" describes the type of the old individual being replaced, not the type of the new offspring.

For the five-individual example, the replacement chances are $2/5 = 0.4$ for A and $3/5 = 0.6$ for B. They remain these values regardless of which type was chosen as the parent. We have not removed anyone between the draws.

## 3.4 List all four parent–replacement combinations

There are two parent types and two replacement types, giving four possible combinations. Conditional on the current state, the two draws are independent. We therefore multiply the appropriate parent and replacement probabilities.

| Parent | Old individual replaced | Probability | A-count change |
|---|---|---|---|
| A | A | $\frac{ri}{W_i}\frac{i}{N}$ | 0 |
| A | B | $\frac{ri}{W_i}\frac{N-i}{N}$ | +1 |
| B | A | $\frac{N-i}{W_i}\frac{i}{N}$ | −1 |
| B | B | $\frac{N-i}{W_i}\frac{N-i}{N}$ | 0 |

If A produces an offspring that replaces B, the population gains one A. If B produces an offspring that replaces A, the population loses one A. If parent and replacement have the same type, the count is unchanged.

The two unchanged cases include more than replacement of the parent itself. An A parent can replace a different A individual too. We only record type counts, so both possibilities have the same effect on our state.

## 3.5 Name the three transition probabilities

A **transition** is a move from the current state to the next state. Let $T_i^+$ mean the probability of moving from $i$ to $i+1$. Let $T_i^-$ mean the probability of moving from $i$ to $i-1$. Let $T_i^0$ mean the probability of remaining at $i$.

The superscripts $+$, $-$, and 0 are labels for increase, decrease, and no change. They are not exponents in this notation.

Reading the four-case table gives

$$T_i^+ = \frac{ri}{ri+N-i}\frac{N-i}{N}, \tag{3.1}$$

$$T_i^- = \frac{N-i}{ri+N-i}\frac{i}{N}, \tag{3.2}$$

$$T_i^0 = 1 - T_i^+ - T_i^-. \tag{3.3}$$

The last equation follows because these three outcomes are mutually exclusive and cover every possibility. It is also the sum of the A–A and B–B probabilities in the table.

### 3.5.1 Check that the probabilities make sense

Every factor in the first two expressions is a nonnegative probability. The four-case table contains all possible results of the two draws, so its entries add to one. Therefore $T_i^0$ is nonnegative too, and

$$T_i^+ + T_i^- + T_i^0 = 1.$$

At $i = 0$, the numerator of each change probability contains a factor $i = 0$. Thus $T_0^+ = T_0^- = 0$ and $T_0^0 = 1$. At $i = N$, the factor $N - i = 0$ gives the same conclusion. The two boundaries really are absorbing under these rules.

## 3.6 Work through the advantageous example completely

Return to $N = 5$, $i = 2$, and $s = 0.5$, so $r = 1.5$ and $W_i = 6$. The parent chances are 0.5 for each type. The replacement chances are 0.4 for A and 0.6 for B. The four joint probabilities are

$$\begin{aligned}
&\Pr(\text{A parent, A replaced} \mid i = 2) = 0.5(0.4) = 0.2,\\
&\Pr(\text{A parent, B replaced} \mid i = 2) = 0.5(0.6) = 0.3,\\
&\Pr(\text{B parent, A replaced} \mid i = 2) = 0.5(0.4) = 0.2,\\
&\Pr(\text{B parent, B replaced} \mid i = 2) = 0.5(0.6) = 0.3.
\end{aligned}$$

Consequently,

$$T_2^+ = 0.3, \qquad T_2^- = 0.2, \qquad T_2^0 = 0.2 + 0.3 = 0.5.$$

The expected count change is $1(0.3) + (-1)(0.2) + 0(0.5) = 0.1$. Selection pushes the average upward. The next event can still decrease the count, and in this state it does so with probability 0.2.

### 3.6.1 Follow two particular random events

One way to perform the two selections is to draw two independent uniform random numbers. A uniform random number between zero and one assigns a probability equal to its length to each interval. For example, the interval from zero to 0.5 has probability 0.5.

Let $a$ denote the number used for the parent draw, and let $b$ denote the number used for the replacement draw. In this state, choose an A parent if $a < 0.5$, and choose an A replacement if $b < 0.4$. Otherwise choose B for the corresponding selection.

**First event, starting with two A individuals.** Suppose $a = 0.2$ and $b = 0.8$. Since $0.2 < 0.5$, the parent is A. Since 0.8 is not less than 0.4, the old individual selected for replacement is B. The A offspring replaces B. The count changes from two to three.
**Second example, resetting to two A individuals.** Suppose $a = 0.8$ and $b = 0.2$. The parent is B because 0.8 is not less than 0.5. The replacement is A because $0.2 < 0.4$. The count changes from two to one.
We reset the starting count between these examples. If we continued directly from the first event instead, we would need to recompute the probabilities using a count of three.

### 3.6.2 A shorter but equivalent implementation

Since we only record the count, the program does not need to generate parent and replacement labels separately. Let $z$ denote one uniform random number between zero and one. We can divide its possible range into three intervals:

- If $z < T_i^+$, increase the count by one.
- Otherwise, if $z < T_i^+ + T_i^-$, decrease the count by one.
- Otherwise, leave the count unchanged.

For the example, the interval lengths are 0.3, 0.2, and 0.5. This reproduces the same three outcome probabilities. It is not changing the biology of the model; it is sampling the already calculated outcomes more directly.

## 3.7 Repeat the calculation when selection is absent

Set $s = 0$, so $r = 1$. The total reproductive weight becomes $W_i = i + N - i = N$. Both change probabilities simplify to

$$T_i^+ = T_i^- = \frac{i(N-i)}{N^2}.$$

The symbol $N^2$ means $N$ multiplied by itself. For $N = 5$ and $i = 2$, this gives

$$T_2^+ = T_2^- = \frac{2(5-2)}{5^2} = \frac{6}{25} = 0.24,$$

and

$$T_2^0 = 1 - 0.24 - 0.24 = 0.52.$$

The expected count change is zero because increases and decreases have equal probability. Nevertheless, nearly half of all event attempts change the count in this state. Neutrality means no systematic reproductive advantage; it does not mean no evolution by drift.

## 3.8 Why we keep unchanged events

Imagine removing every no-change event from a recorded trajectory. The sequence of states at the remaining changes could still tell us which boundary was reached. But the number of recorded steps would no longer be the number of reproduction and replacement attempts.

The simulation in this project keeps all attempts. A plotted horizontal segment is a real feature of the recorded count process. It is not an error or an instruction that failed to run.

**Question.** A neutral population has $N = 4$ and $i = 1$. Work out both parent probabilities, both replacement probabilities, and the three transition probabilities. What is the expected count change?
**Worked answer.** Neutrality means $r = 1$, so the total parent weight is four. An A parent has probability $1/4$, and a B parent has probability $3/4$. The replacement probabilities are also $1/4$ for A and $3/4$ for B. Therefore

$$T_1^+ = \frac{1}{4}\frac{3}{4} = \frac{3}{16}, \qquad T_1^- = \frac{3}{4}\frac{1}{4} = \frac{3}{16}.$$

The no-change probability is $1 - 3/16 - 3/16 = 10/16$. The expected count change is $3/16 - 3/16 = 0$. A change is possible despite the zero expectation.

The next task is to express these calculations as instructions that Julia can execute. The mathematical inputs are just $N$, $i$, and $s$. The immediate output is the probability of each possible next count.

# Chapter 4

# The Julia you need, starting from the beginning

## 4.1 What is a program?

A program is a written set of instructions for a computer. Julia is the language in which we write those instructions. A Julia file is an ordinary text file whose name usually ends in `.jl`. You can open it, read it, change it, and run it. Reading a file does not run it.

Our program will not contain a tiny picture of every organism. It will store a number: how many individuals currently carry the focal allele. That is enough because our rules treat individuals of the same type identically. This is a deliberate simplification of the biology, not a limitation of the computer.

## 4.2 Two places where you can type commands

The *terminal* is an application that accepts commands for your computer. On a Mac it may show a prompt ending with a dollar sign or a percent sign. Do not copy the prompt itself. The Julia *REPL* is an interactive session that accepts Julia expressions. Its prompt is `julia>`. REPL means read, evaluate, print, loop: it reads your instruction, calculates the result, prints it, and waits for another instruction.

> In the terminal, type `julia`. When you see `julia>`, type `2 + 3`. Julia prints `5`. Type `exit()` to leave Julia and return to the terminal. A terminal command such as `julia scripts/demo.jl` belongs in the terminal, not after the `julia>` prompt.

The commands in this guide assume that the terminal's current directory is the project folder. A directory is a folder. The `cd` command changes directory. If the folder path contains spaces, put quotation marks around the whole path. To avoid needing to guess your download location, you can type `cd ` with a trailing space, drag the extracted project folder into the terminal, and press Return.

```
julia --version
julia --project=. scripts/demo.jl
```

The first command prints the installed Julia version. The second starts Julia, uses the project in the current directory (the dot means "here"), and runs a file. The text `-project=.` is an option to Julia, not a line of Julia code. You do not need to type `julia>` anywhere in either command.

## 4.3 Names store values

```
N = 5
i = 2
s = 0.5
x = i / N
```

Read the first line as "store the number five under the name N." The equals sign assigns a value. It does not ask a question. The second line stores the allele count, the third stores the selective advantage, and the fourth calculates the frequency. The slash means division. Julia prints `0.4` if you ask for `x`.

An integer is a whole number such as `5`. A floating-point number represents a number with a fractional part, such as `0.5`. Most such numbers are stored with finite precision. This becomes relevant much later when we calculate probabilities very close to zero. For now, ordinary arithmetic works as you expect.

| Julia expression | Meaning |
|---|---|
| `1 + s` | add one and the advantage |
| `r * i` | multiply fitness and count |
| `N - i` | subtract the focal count from the total |
| `i / N` | divide count by total |
| `r^2` | square the fitness |
| `(1 + s) * i` | do the addition before multiplying |

Multiplication needs a star. Write `r * i`, not `ri`. In this project a name such as `total_weight` is one name, not a subtraction. The underscore helps people read it.

## 4.4 Comments explain; they do not calculate

A hash sign starts a comment. Julia ignores everything after it on that line.

```
x = i / N  # Frequency is count divided by population size.
```

Comments tell a reader what a calculation means. They do not make a wrong calculation correct. We will check the calculations separately.

## 4.5 Functions are reusable instructions

```
function allele_frequency(i, N)
```

```
    x = i / N
    return x
end

allele_frequency(2, 5)
```

1. `function` begins a reusable instruction. `allele_frequency` is the name we choose for it.
2. Inside parentheses, `i` and `N` name the inputs. Inputs are also called arguments.
3. The next line performs the calculation using those inputs.
4. `return x` sends the answer back to the place that called the function.
5. `end` closes the definition. Defining a function does not yet carry out the calculation for a particular population.
6. The final line *calls* the function: it supplies two and five as the inputs. Its answer is `0.4`.

The order of arguments matters. `allele_frequency(5, 2)` asks for a different calculation. In our supplied functions the population size comes first; always check the function's documentation rather than guessing from this introductory example.

Names created inside a function normally belong to that function. A variable called `i` in one function does not automatically alter an unrelated `i` elsewhere. This helps keep a program understandable.

## 4.6 Ask questions with comparisons

```
i == 0
i == N
i < N
0 <= i <= N
```

Each expression returns `true` or `false`. Two equals signs ask whether values are equal. One equals sign assigns a value. Confusing these is a common early mistake. The last expression checks whether the count is between zero and the population size, including both endpoints.

```
if i == 0
    println("The allele has been lost.")
elseif i == N
    println("The allele has fixed.")
else
    println("Both types are still present.")
end
```

Julia tests the first question. If it is true, it runs the first message. Otherwise it tries the second question. If neither is true, it runs the `else` branch. Only one message is printed. `println` displays text and then starts a new output line. Quotation marks mark text; the messages do not affect the allele count.

## 4.7 Loops repeat instructions

```
for repetition in 1:3
    println(repetition)
end
```

The colon creates the sequence one, two, three. The loop takes each number in turn, stores it in `repetition`, and runs its body. The output is three lines containing `1`, `2`, and `3`. The name `repetition` is descriptive; it is not a special Julia word.

A `while` loop repeats while a condition is true. In the model, `while 0 < i < N` means "keep reproducing while both types are present." If an event does not change `i`, the loop still carried out an event. It must test the condition and try another event. A simulation can therefore take many iterations.

## 4.8 Arrays store several values in order

```
counts = [2, 3, 3, 2]
counts[1]
push!(counts, 1)
length(counts)
```

Square brackets create an array, an ordered collection of values. In Julia the first entry is numbered one. Therefore `counts[1]` is two. The command `push!` adds a new entry to the end. Our array becomes `[2, 3, 3, 2, 1]`, and its length is five. A trailing exclamation mark is a Julia naming convention warning that a function changes something supplied to it. It is part of the function's name.

If the initial state is recorded, then after four reproduction events there are five recorded counts. Entry one is the state at event zero. Keep this one-entry difference in mind when reading a trajectory file.

## 4.9 Random numbers select events

```
using Random
rng = MersenneTwister(20261027)
u = rand(rng)
```

`Random` is a standard Julia library. `using Random` makes its tools available. `MersenneTwister` creates a pseudorandom-number generator: a mathematical procedure producing numbers that behave like random draws for our simulation. The integer in parentheses is its seed, a starting setting. `rand(rng)` takes the next number from that generator. Here the result is at least zero and less than one.

Imagine a line from zero to one divided into three sections of lengths 0.3, 0.2, and 0.5. A uniform draw lands in each section with the corresponding probability. We can therefore use its position to choose an increase, decrease, or no change.

Setting a seed once makes a run reproducible in the recorded software environment. Setting the

same seed before every repetition would repeatedly restart the same random sequence, which does *not* produce independent repetitions. We keep drawing from the generator instead.

The demonstration uses some chosen numbers so that you can check the arithmetic by hand. These are teaching inputs, not evidence from a random experiment. The statistical experiments use actual draws from the generator.

## 4.10 More than one answer from a function

A *named tuple* groups a few values and gives each a name. The model's transition function returns three probabilities. A result with fields `up`, `down`, and `stay` lets us write `probabilities.up`. The dot asks for the value stored under that field name. This is easier to read than remembering that the first entry means an increase.

## 4.11 Files, libraries, and the project

`include` reads and evaluates a Julia source file. A `module` gives a related set of functions its own name. The supplied source is called `MoranModel`; after including it, `using .MoranModel` makes its exported functions available. The dot means the module was defined locally rather than installed as an external package.

The project uses standard libraries supplied with Julia. Its `Project.toml` identifies these dependencies; `Manifest.toml` records the resolved environment. A TOML file is a configuration file, not a Julia program. The README gives the exact commands, so you do not need to learn configuration syntax to begin.

**Try it.** What are the values of `i`, `N-i`, and `i/N` after `N = 10` and `i = 3`? Is `i == N` true?

**Answer.** The count is three, the other type has seven individuals, and the frequency is 0.3. The equality test is false: three is not ten. Nothing in that test changes either variable.

**Try it.** Why can the first recorded state in an array be numbered one even though it represents event zero?

**Answer.** Array position and biological event number are different labels. Julia numbers positions from one. We choose to record the population before any event, so that state occupies position one and has event number zero.

# Chapter 5

# Read the actual model code, one step at a time

## 5.1 The map before the details

The file `src/MoranModel.jl` contains the rules and calculations. It does not run the whole study simply because you open it. A script in the `scripts` folder loads those rules and calls the functions. This chapter explains the model file. Chapter 10 explains the scripts that organise the experiments and save the results.

There are nine functions. First we reject impossible inputs. Next we calculate the probabilities for one event. Then we turn a random number into an event. Those three pieces let us build a complete history and repeat the experiment. The remaining functions calculate an exact answer, check that answer another way, calculate a confidence interval, and produce a deterministic comparison.

The listings below use byte-for-byte copies of the runnable files, refreshed by the guide-preparation script. Keeping these copies inside the guide folder allows the LaTeX editor to compile a self-contained project. The numbers at the left are its actual line numbers. Appendix B contains the complete file without interruptions.

```
1 module MoranModel
2
3 using Random
4 using LinearAlgebra
5
6 export validate_parameters, transition_probabilities, next_count
7 export simulate_trajectory, simulate_fixation, fixation_probability
8 export backward_probabilities, wilson_interval, deterministic_trajectory
```

Line 1 begins a module named `MoranModel`. Lines 3–4 load Julia's standard tools for random numbers and linear algebra. Linear algebra is a way of solving several linked equations; we explain our small example in Chapter 7. Lines 6–8 export the names of our functions, making them convenient to call after loading the module. The final `end` at the bottom of the file closes this module. It is separate from the `end` that closes each function.

## 5.2 Function 1: reject inputs that do not describe our model

```
11 function validate_parameters(N, i, s)
12     N isa Integer && N >= 2 || throw(ArgumentError("N must be an integer at least 2."))
13     i isa Integer && 0 <= i <= N || throw(ArgumentError("i must be an integer from 0 to N."))
14     s isa Real && isfinite(s) && s >= 0 && isfinite(Float64(s)) && (s == 0 || Float64(s) > 0)
         || throw(ArgumentError("s must be finite and nonnegative; a positive s must convert to a
         finite, positive Float64."))
15     return nothing
16 end
```

**Line 11**

Define a function receiving population size `N`, focal count `i`, and selective advantage `s`.

**Line 12**

`isa Integer` checks for a whole-number input. The double ampersand means "and." Both the integer check and `N >= 2` must be true. The double vertical bar means "or": if the preceding condition fails, run `throw`, which stops with a descriptive error. `ArgumentError` names the kind of error. The biological requirement is a population of at least two individuals.

**Line 13**

Apply the same idea to the allele count. Zero is allowed because the allele may already be lost; `N` is allowed because it may already be fixed.

**Line 14**

Require a finite real number at least zero. `isfinite` excludes infinity and undefined numerical values. A positive input must also remain finite and positive when converted to `Float64`; we reject numbers outside this implementation's representable range. Negative selection is outside this teaching project's scope.

**Line 15**

`nothing` is Julia's value meaning that there is no ordinary result to return. This function checks inputs; it does not calculate a probability.

**Line 16**

Close the function. A passed check allows the caller to continue; a failed check prevents it from silently producing an answer for the wrong model.

## 5.3 Function 2: calculate the three event probabilities

```
19 function transition_probabilities(N, i, s)
20     validate_parameters(N, i, s)
21     relative_fitness = 1.0 + Float64(s)
22     # Dividing both reproductive weights by relative_fitness avoids overflow.
23     weight_A = Float64(i)
24     weight_B = (N - i) / relative_fitness
25     total_weight = weight_A + weight_B
26     parent_A = weight_A / total_weight
27     parent_B = weight_B / total_weight
28     up = parent_A * (N - i) / N
29     down = parent_B * i / N
```

```
30      stay = 1.0 - up - down
31      return (up=up, down=down, stay=stay)
32  end
```

The mathematical derivation used total reproductive weight $(1+s)i+(N-i)$. The code uses a harmless rescaling. Dividing *every* individual's reproductive weight by the same positive number does not change which type is more likely to reproduce. In particular,

$$\frac{(1+s)i}{(1+s)i+(N-i)} = \frac{i}{i+(N-i)/(1+s)}.$$

We divided both the top and the bottom by $1+s$. For $N=5, i=2, s=0.5$, the original weights are 3 and 3; the scaled weights are 2 and 2. Either calculation gives a parent probability of one half.

**Lines 19–20**
Begin the function and check its inputs.

**Line 21**
Calculate $1+s$. `Float64` converts the input to Julia's ordinary double-precision floating-point representation. The `1.0` is a floating-point one.

**Line 22**
Explain the rescaling in a comment. It avoids multiplying a large fitness by the count.

**Line 23**
Give each focal individual scaled weight one, so their combined weight is `i`.

**Line 24**
Give each other individual scaled weight $1/(1+s)$, so their combined weight is $(N-i)/(1+s)$.

**Line 25**
Add the two weights to find the total.

**Lines 26–27**
Divide each type's weight by the total. These are parent probabilities, not probabilities of changing the count.

**Line 28**
To increase the count, a focal parent must reproduce *and* an individual of the other type must be replaced. Multiply the first probability by $(N-i)/N$.

**Line 29**
To decrease the count, the other type must reproduce and a focal individual must be replaced. Multiply the relevant probabilities.

**Line 30**
Whatever probability remains is the chance of no change. The three outcomes cover every possible event.

**Lines 31–32**
Return a named tuple and close the function. In `up=up`, the name on the left labels the returned field; the value on the right is the quantity just calculated.

## 5.4 Function 3: use a random number to choose the next count

```
35 function next_count(i, up, down, u)
36     0 <= u < 1 || throw(ArgumentError("u must be at least 0 and less than 1."))
37     if u < up
38         return i + 1
39     elseif u < up + down
40         return i - 1
41     else
42         return i
43     end
44 end
```

**Lines 35–36**
Receive the current count, the two change probabilities, and a number `u`. Check that the number lies between zero, inclusive, and one, exclusive.

**Lines 37–38**
If `u` is below the increase probability, return a count larger by one.

**Lines 39–40**
Otherwise test whether `u` lies below the combined increase-and-decrease probability. The first branch has already excluded the increase section, so this branch corresponds only to a decrease.

**Lines 41–42**
All remaining draws mean no change. Return the original count.

**Lines 43–44**
Close the conditional and the function. Each call returns exactly one integer.

For probabilities $0.3, 0.2, 0.5$, draws in $[0, 0.3)$ increase the count, draws in $[0.3, 0.5)$ decrease it, and draws in $[0.5, 1)$ preserve it. Square brackets include an endpoint; round brackets exclude an endpoint. Draws 0.12, 0.41, and 0.72 therefore produce three different outcomes from the *same initial state*.

**A sequence needs new probabilities after every event.** The demonstration starts at $i = 2$ and uses the numbers $0.12, 0.72, 0.41, 0.95, 0.10, 0.03$. Its successive counts are $3, 3, 2, 2, 3, 4$. After the first increase, the population has changed, so the program recalculates its probabilities before using the next draw. Do not apply $0.3, 0.2, 0.5$ unchanged to all six events.

## 5.5 Function 4: record an entire population history

```
47 function simulate_trajectory(rng, N, i0, s, max_events)
48     validate_parameters(N, i0, s)
49     max_events isa Integer && max_events >= 0 || throw(ArgumentError("max_events must be a
       nonnegative integer."))
50     counts = Vector{Int}(undef, max_events + 1)
51     counts[1] = i0
52     i = i0
53     absorbed_at = nothing
```

```
54      if i == 0 || i == N
55          absorbed_at = 0
56      end
57      for event in 1:max_events
58          if 0 < i < N
59              probabilities = transition_probabilities(N, i, s)
60              i = next_count(i, probabilities.up, probabilities.down, rand(rng))
61              if i == 0 || i == N
62                  absorbed_at = event
63              end
64          end
65          counts[event + 1] = i
66      end
67      return (counts=counts, absorbed=absorbed_at !== nothing, absorbed_at=absorbed_at)
68  end
```

**Lines 47–49**

Receive the random-number generator, population inputs, and the maximum number of events to display. Check both the biological inputs and the nonnegative whole-number horizon. `i0` means the initial count; it is a name, not $i$ multiplied by zero.

**Line 50**

Allocate storage for integer counts. `Vector{Int}` means an array of integers. `undef` means the entries are not filled in yet. There are `max_events + 1` entries because we also record the initial state.

**Lines 51–52**

Store the initial count in entry one and copy it into the variable that will track the changing population.

**Line 53**

Initially no absorption event has been recorded. `nothing` is different from zero: zero would mean absorption already occurred at the initial state.

**Lines 54–56**

If the initial state is already lost or fixed, record absorption at event zero. The double vertical bar inside the `if` condition means "or."

**Line 57**

Begin a loop over every event up to the display horizon.

**Lines 58–60**

If both types are present, calculate this state's probabilities and call `next_count` with a fresh draw. The returned count replaces the old value of `i`.

**Lines 61–63**

If this event reaches zero or `N`, record the event number. Absorption means the count will remain there in this mutation-free model.

**Line 64**

Close the condition that permits reproduction updates. If the population was already absorbed, no further random draws were needed.

**Line 65**

Store the count. After absorption we keep storing the same value, so every displayed path

has the same length. This does not change the time of absorption already recorded.

**Lines 66–68**
Finish the loop and return the array, an absorption flag, and the absorption time. `!== nothing` asks whether a time has actually been recorded.

A display horizon is not a rule for deciding fixation. If a path still contains both types at its horizon, it is unfinished. Our separate fixation experiment below has no such horizon.

## 5.6 Function 5: run until loss or fixation

```
71 function simulate_fixation(rng, N, i0, s)
72     validate_parameters(N, i0, s)
73     i = i0
74     events = 0
75     while 0 < i < N
76         probabilities = transition_probabilities(N, i, s)
77         i = next_count(i, probabilities.up, probabilities.down, rand(rng))
78         events += 1
79     end
80     return (fixed=i == N, events=events)
81 end
```

**Lines 71–73**
Define the function, check the inputs, and initialise the current count.

**Line 74**
Start an event counter at zero.

**Line 75**
Continue while the count lies strictly between the two absorbing boundaries.

**Lines 76–77**
Recalculate the probabilities and use a fresh random draw to update the state.

**Line 78**
Add one to the number of events. `events += 1` is shorthand for `events = events + 1`. It also counts events that did not change the allele count.

**Line 79**
Return to the loop condition. Exit only after reaching a boundary.

**Lines 80–81**
Return whether the final state is fixation and how many events elapsed. `fixed=i == N` labels the field `fixed` with a true-or-false answer. There is no cutoff that could mistake a surviving allele for a lost allele.

This function stores only the current state. It does not store all previous states because a fixation experiment needs the final outcome rather than a graph of the whole history. That makes many repetitions use little memory.

## 5.7 Function 6: calculate the exact answer accurately

```
84 function fixation_probability(N, i, s)
85     validate_parameters(N, i, s)
86     if i == 0
87         return 0.0
88     elseif i == N
89         return 1.0
90     elseif s == 0
91         return i / N
92     end
93     log_fitness = log1p(Float64(s))
94     numerator = -expm1(-i * log_fitness)
95     denominator = -expm1(-N * log_fitness)
96     return numerator / denominator
97 end
```

Chapter 7 derives the answer. You may read this section once now to see how the code is organised and return after deriving it.

**Lines 84–85**
Define the exact-probability function and check the inputs.

**Lines 86–89**
A lost allele has fixation probability zero; an already fixed allele has probability one. Return these exact boundary answers immediately.

**Lines 90–92**
With no selection, return $i/N$. Handling neutrality separately avoids trying to divide zero by zero in the general expression.

**Line 93**
Calculate $\log(1+s)$ using `log1p`. The natural logarithm reverses exponentiation by the constant $e$; Chapter 8 introduces it. The specialised function preserves small differences when $s$ is tiny.

**Line 94**
Calculate $1-\exp[-i\log(1+s)]$. Julia's `expm1(y)` computes $e^y-1$ accurately when $y$ is close to zero. The leading minus changes its sign, giving $1-e^y$.

**Line 95**
Calculate the same expression using $N$ in place of $i$.

**Lines 96–97**
Divide the two quantities and return the exact fixation probability, up to floating-point rounding.

The computer expression is the same formula as

$$\frac{1-(1+s)^{-i}}{1-(1+s)^{-N}}.$$

Its appearance differs to protect numerical accuracy. If the computer rounds two very similar numbers to the same stored number, their difference can lose useful digits. This is called cancellation. The specialised functions avoid unnecessary cancellation; they do not add a new biological assumption.

## 5.8 Function 7: check the exact formula by another method

```
100 function backward_probabilities(N, s)
101     validate_parameters(N, 0, s)
102     diagonal = zeros(N - 1)
103     lower = zeros(N - 2)
104     upper = zeros(N - 2)
105     right_hand_side = zeros(N - 1)
106     for i in 1:(N - 1)
107         probabilities = transition_probabilities(N, i, s)
108         diagonal[i] = probabilities.up + probabilities.down
109         if i > 1
110             lower[i - 1] = -probabilities.down
111         end
112         if i < N - 1
113             upper[i] = -probabilities.up
114         else
115             right_hand_side[i] = probabilities.up
116         end
117     end
118     matrix = Tridiagonal(lower, diagonal, upper)
119     interior = matrix \ right_hand_side
120     return vcat(0.0, interior, 1.0)
121 end
```

The backward recurrence links the unknown fixation probability in state $i$ with those in states $i-1$ and $i+1$. Rearranged, it is

$$(T_i^+ + T_i^-)u_i - T_i^- u_{i-1} - T_i^+ u_{i+1} = 0.$$

Here $u_i$ is the probability we are trying to find, and $T_i^+, T_i^-$ are the event probabilities. The boundary answers $u_0 = 0, u_N = 1$ are known. The remaining $N-1$ answers must satisfy $N-1$ linked equations. Chapter 7 solves a two-unknown example by ordinary substitution.

**Lines 100–101**
Define the function and validate population size and selection. We use zero as a valid placeholder count for the shared checker.

**Lines 102–105**
Create arrays of zeros for the coefficients of the unknowns and the constants on the right side. `zeros` stores floating-point zeros. The main diagonal has $N-1$ entries; each adjacent diagonal has $N-2$.

**Line 106**
Visit each nonabsorbing state $i = 1, \ldots, N-1$.

**Lines 107–108**
Get the event probabilities and store the coefficient of $u_i$.

**Lines 109–111**
If a lower interior neighbour exists, store the coefficient of $u_{i-1}$. For state one the lower neighbour is the known zero boundary, so no unknown entry is required.

**Lines 112–116**
Store the coefficient of the upper interior neighbour when it exists. At the last interior

state, the upper neighbour is $u_N = 1$. Move its term to the right side, giving a positive constant $T_i^+$.

**Lines 117–118**
Finish filling the coefficients and assemble a tridiagonal matrix: a compact table with nonzero entries only on the central diagonal and its two neighbours.

**Line 119**
Julia's backslash asks its linear-algebra tools to solve the linked equations. It is not ordinary division. The vector `interior` holds the unknown probabilities in state order.

**Lines 120–121**
`vcat` joins values vertically. Attach the known zero and one boundary values to the interior answers and return all $N + 1$ probabilities.

This calculation does not use the closed-form fixation formula. Agreement checks two different ways of solving the same specified model. It is stronger evidence than running the same formula twice, although it does not prove the model is a realistic description of a particular organism.

## 5.9 Function 8: calculate uncertainty from repeated experiments

```
124 function wilson_interval(k, n)
125     n isa Integer && n > 0 || throw(ArgumentError("n must be a positive integer."))
126     k isa Integer && 0 <= k <= n || throw(ArgumentError("k must be an integer from 0 to n."))
127     z = 1.959963984540054
128     estimate = k / n
129     denominator = 1.0 + z^2 / n
130     centre = (estimate + z^2 / (2n)) / denominator
131     half_width = z * sqrt(estimate * (1 - estimate) / n + z^2 / (4n^2)) / denominator
132     lower = max(0.0, centre - half_width)
133     upper = min(1.0, centre + half_width)
134     if k == 0
135         lower = 0.0
136     elseif k == n
137         upper = 1.0
138     end
139     return (lower=lower, upper=upper)
140 end
```

Chapter 6 explains this calculation with numbers before asking you to interpret intervals in the results.

**Lines 124–126**
Accept `k` fixations in `n` independent repetitions. Require a positive integer number of repetitions and a fixation count from zero through `n`.

**Line 127**
Set the normal-distribution multiplier for a two-sided 95% interval, approximately 1.96.

**Line 128**
Calculate the observed proportion $k/n$.

**Line 129**

Calculate a denominator shared by the next two expressions.

**Line 130**

Calculate the Wilson interval's centre. `2n` is Julia's permitted shorthand for `2 * n`.

**Line 131**

Calculate the distance from that centre to either endpoint. `sqrt` means square root. `4n^2` means $4n^2$, not $(4n)^2$.

**Lines 132–133**

Subtract and add the half-width. `max` and `min` keep the numerical endpoints within zero and one.

**Lines 134–138**

Explicitly set the zero-success lower endpoint to zero and the all-success upper endpoint to one. This also removes insignificant floating-point rounding at those boundaries.

**Lines 139–140**

Return the two named endpoints. An interval is not a guarantee that this particular simulation estimate equals the exact answer.

## 5.10 Function 9: calculate a deterministic comparison

```
143 function deterministic_trajectory(N, i0, s, max_events)
144     validate_parameters(N, i0, s)
145     max_events isa Integer && max_events >= 0 || throw(ArgumentError("max_events must be
    nonnegative."))
146     frequencies = Vector{Float64}(undef, max_events + 1)
147     x = i0 / N
148     frequencies[1] = x
149     for event in 1:max_events
150         change = (s * x / (1.0 + s * x)) * ((1.0 - x) / N)
151         x = x + change
152         frequencies[event + 1] = x
153     end
154     return frequencies
155 end
```

**Lines 143–145**

Receive population inputs and a horizon and check them.

**Line 146**

Allocate an array of floating-point frequencies, with room for the initial value.

**Lines 147–148**

Convert the initial count into a frequency and store it.

**Line 149**

Step through the same number of events as the stochastic display.

**Line 150**

Calculate the expected one-event frequency change evaluated at the current deterministic frequency. Multiplying out the expression gives $sx(1-x)/[N(1+sx)]$.

**Line 151**

Add this change to the current frequency. There is no random draw.

**Lines 152–155**

Store the updated frequency, continue the loop, and return the array.

At $N = 5, i = 2, s = 0.5$, the first change is $(0.3 - 0.2)/5 = 0.02$. The deterministic frequency changes from 0.40 to 0.42. A real model event can only change it to 0.60, 0.20, or leave it at 0.40. The deterministic number is an expectation-based comparison, not a possible outcome of that one event. Chapter 8 explains why repeatedly applying it is a mean-field approximation.

**Try it.** Why do we increment the event counter after a no-change event? Why can the fixation function avoid keeping an array?

**Answer.** Reproduction and replacement still occurred, so one model event elapsed. The count happened to remain the same. For a fixation experiment we need only whether the allele eventually fixed and how long the experiment took; earlier counts are unnecessary unless we want to draw a history.

# Chapter 6

# Learning from repeated experiments

## 6.1 One history cannot tell us a probability

A single run of the model produces one history. The focal allele might disappear. It might eventually occupy the whole population. Watching that one history does not tell us how often either outcome occurs.

Think about tossing a coin. Seeing one head does not establish that every future toss will be a head. To learn about the chance of a head, we repeat the experiment. We use the same idea here, except that one experiment is an entire population history, possibly containing thousands of reproduction events.

A *replicate* means one fresh repetition of the whole experiment. Every replicate starts from the same population size, initial allele count and fitness advantage. It uses a fresh part of the random-number sequence. It continues until the allele is lost or fixes.

Imagine running ten populations, each starting with one copy of the focal allele. Suppose the outcomes, in order, are

lost, lost, fixed, lost, lost, lost, lost, fixed, lost, lost.

There are two fixation outcomes out of ten runs. Our estimate of the fixation probability is

$$\frac{\text{number of fixations}}{\text{number of runs}} = \frac{2}{10} = 0.2.$$

This is an invented small example to explain the calculation. It is not a result from the full study.

We will use three symbols. The capital letter $R$ is the number of replicates. The lower-case letter $k$ is the number of those replicates that fix. The symbol $\widehat{p}$, read "p hat", is the estimated fixation probability:

$$\widehat{p} = \frac{k}{R}. \tag{6.1}$$

The hat tells us that this number was estimated from a finite set of runs. We use $p$, without a hat, for the true fixation probability under the model. Usually a simulation study does not know $p$ in advance. Our model has an exact formula, which gives us a valuable way to check the simulation.

## 6.2 What the repetition loop does

The repetition loop is conceptually simple:

1. Set a fixation counter to zero.
2. Start a new population in the initial state.
3. Simulate until loss or fixation.
4. If fixation occurred, add one to the counter.
5. Repeat until $R$ populations have finished.
6. Divide the final counter by $R$.

The *outer* loop runs the independent experiments. Inside it, the simulation function contains an *inner* loop that runs reproduction events for one experiment. Confusing these two loops is a common beginner mistake: twenty thousand events in one population are not twenty thousand independent populations.

Do not declare a long-running history to be a loss merely because it has not fixed yet. An unfinished history has an unknown eventual outcome. Our fixation experiments run until one of the two absorbing states is reached. We may use a shorter time window when drawing example histories, but that window is a display choice, not a rule for classifying fixation.

## 6.3 Random seeds: repeatable does not mean identical trials

A computer normally uses a *pseudorandom-number generator*. This is a rule that produces a long sequence of numbers that behaves sufficiently like independent random draws for this simulation. A *seed* sets its starting state.

If we use the same generator, seed, Julia version and code, we can reproduce the same sequence. This helps us check a calculation. It does not mean that successive replicates receive the same numbers: the generator moves forward each time a draw is requested.

Set the seed once before a batch of experiments. Do *not* reset the generator to the same seed before every replicate. Doing that could make every replicate repeat the same history, creating an illusion of many experiments while adding no independent information.

Our study records a seed for each parameter combination. It also records the Julia version. Keeping these details with the results makes the calculation reproducible. Bit-for-bit random sequences are not promised across arbitrary changes of software version.

**Question.** You reset the generator to seed 42, run a population, reset it to seed 42 again, and run the same population again. Have you obtained two useful independent trials?
**Worked answer.** No. With unchanged code and environment, both runs consume the same sequence of random numbers and produce the same history. Instead, create the generator once and pass that continuing generator into each replicate.

## 6.4 Why estimates change from one batch to another

Even a correct simulation produces slightly different estimates when we change its seed. That difference is *Monte Carlo error*: uncertainty caused by using a finite number of random experiments. "Monte Carlo" here simply means estimating something by random sampling.

Monte Carlo error is different from *model uncertainty*. Monte Carlo error asks whether we have run enough repetitions to estimate the probability of *this model*. Model uncertainty asks whether the assumptions of this model adequately describe a biological situation. Increasing $R$ reduces the first problem. It does not remove the second.

We can describe Monte Carlo error mathematically. For replicate $j$, define a number $Y_j$ as follows:

$$Y_j = \begin{cases} 1, & \text{if replicate } j \text{ fixes,} \\ 0, & \text{if replicate } j \text{ is lost.} \end{cases}$$

The subscript $j$ is a label: $Y_1$ describes the first run, $Y_2$ the second, and so on. A random variable with possible values zero and one is called a *Bernoulli random variable*.

The expected value, written $\mathbb{E}[Y_j]$, means its average over many repetitions. It equals each possible value multiplied by its probability:

$$\mathbb{E}[Y_j] = 1 \times p + 0 \times (1-p) = p.$$

Here $1-p$ is the probability of loss, because fixation and loss are the only eventual outcomes in our model.

The *variance* measures squared fluctuations around an expected value. We write it as Var. For a Bernoulli variable, squaring changes neither possible value: $0^2 = 0$ and $1^2 = 1$. Therefore

$$\mathbb{E}[Y_j^2] = p, \qquad \mathrm{Var}(Y_j) = \mathbb{E}[Y_j^2] - \big(\mathbb{E}[Y_j]\big)^2 = p - p^2 = p(1-p).$$

The total number of fixations is $K = Y_1 + \cdots + Y_R$. We use capital $K$ when discussing this count before the experiment is run, and lower-case $k$ for its observed value. For independent replicates, variances add: the fluctuations of one run do not systematically push the others in the same direction. Thus

$$\mathrm{Var}(K) = R\,p(1-p).$$

Our estimate is $\widehat{p} = K/R$. Dividing a quantity by $R$ divides its squared fluctuations by $R^2$. Consequently,

$$\mathrm{Var}(\widehat{p}) = \frac{\mathrm{Var}(K)}{R^2} \tag{6.2}$$

$$= \frac{R\,p(1-p)}{R^2} \tag{6.3}$$

$$= \frac{p(1-p)}{R}. \tag{6.4}$$

The square root of this variance is the *standard error*:

$$\mathrm{SE}(\widehat{p}) = \sqrt{\frac{p(1-p)}{R}}. \tag{6.5}$$

It describes the typical size of the estimate's fluctuations across repeated batches. When $p$ is unknown, one common estimate replaces $p$ with $\widehat{p}$. When checking against our exact solution, we can calculate the standard error using the exact $p$.

Notice the square root. Multiplying the number of replicates by four approximately halves the standard error. Multiplying it by one hundred approximately divides the standard error by ten. More precision costs more computation.

## 6.5 A rare outcome needs enough repetitions

For neutral selection, one initial copy in a population of $N$ individuals fixes with probability $1/N$. In the largest population in our grid, $N = 200$, so

$$p = \frac{1}{200} = 0.005.$$

This is a half-percent fixation chance. With $R = 20{,}000$ independent runs, the expected number of fixations is

$$Rp = 20{,}000 \times 0.005 = 100.$$

It need not be exactly 100. The standard error of the estimated probability is

$$\sqrt{\frac{0.005 \times 0.995}{20{,}000}} \approx 0.000499.$$

That is about 0.05 percentage points. A rough two-standard-error range around the true probability is about $0.005 \pm 0.001$. The absolute discrepancy is small, but $0.001/0.005 = 0.2$, or 20% of the probability itself. For rare events, relative uncertainty can remain noticeable even when the absolute uncertainty looks tiny.

> Suppose we use only 100 replicates when $p = 0.005$. The expected fixation count is $100 \times 0.005 = 0.5$. Many batches will contain no fixations at all. Such a batch does not establish that fixation is impossible. It shows that 100 runs provide limited information about an event this rare.

## 6.6 Confidence intervals: show a range, not just a point

A point estimate such as $\widehat{p} = 0.1$ hides how much sampling information supports it. Ten fixations out of 100 runs and ten thousand fixations out of one hundred thousand runs both give 0.1, but the latter is much more precise.

A *confidence interval* is a procedure that turns the observed count and sample size into a range. We use a *Wilson interval*. It behaves more sensibly than the simplest symmetric interval when counts are small or the estimate is near zero or one.

The interval uses a number $z = 1.96$, a conventional threshold for an approximate 95% interval based on a bell-shaped reference distribution. You do not need to memorise 1.96 to understand what the code does. It is an input specifying the uncertainty level.

Define a denominator $D$, a centre $C$ and a half-width $H$:

$$D = 1 + \frac{z^2}{R}, \tag{6.6}$$

$$C = \frac{\widehat{p} + z^2/(2R)}{D}, \tag{6.7}$$

$$H = \frac{z}{D}\sqrt{\frac{\widehat{p}(1-\widehat{p})}{R} + \frac{z^2}{4R^2}}. \tag{6.8}$$

The lower endpoint is $C - H$ and the upper endpoint is $C + H$. The centre is generally a little different from $\widehat{p}$; that is intentional.

Where does this expression come from? Start with the comparison

$$|\widehat{p} - p| \leq z\sqrt{\frac{p(1-p)}{R}}.$$

The vertical bars mean "absolute difference". Temporarily treat $p$ as an unknown candidate probability. Squaring both sides and multiplying by $R$ gives

$$R(\widehat{p} - p)^2 \leq z^2 p(1-p).$$

Expand the square, bring everything to the left, and collect equal powers of $p$:

$$(R + z^2)p^2 - (2R\widehat{p} + z^2)p + R\widehat{p}^2 \leq 0.$$

This is a quadratic inequality: it contains $p^2$, $p$ and a constant. Its two roots bound the acceptable values of $p$. Applying the quadratic formula gives exactly $C - H$ and $C + H$ above. The confidence level is approximate because the starting comparison uses an approximation to sampling fluctuations.

Take $R = 100$ runs and $k = 10$ fixations. First calculate $\widehat{p} = 10/100 = 0.1$. With $z = 1.96$, the Wilson calculation gives

$$D = 1.038416, \qquad C \approx 0.114798, \qquad H \approx 0.059569.$$

The endpoints are approximately

$$0.0552 \quad \text{and} \quad 0.1744.$$

We would report an estimated fixation probability of 0.10, with a 95% Wilson interval of about 0.055 to 0.174. These numbers illustrate the method; they do not come from the full parameter study.

The 95% statement describes the procedure across repeated batches: approximately 95% of intervals produced in this way would contain the fixed model probability under suitable sampling conditions. It does not mean that, after calculating these endpoints, the fixed probability has a 95% chance of lying inside them. Exact coverage can differ from 95% because counts are discrete and the method is approximate.

If we observe zero fixations, the Wilson interval still has a positive upper endpoint. That is appropriate: observing no successes in a finite sample is compatible with a small positive probability.

## 6.7 How this applies to our full study

Our study has four population sizes and seven selection coefficients. That produces $4 \times 7 = 28$ parameter combinations. Each receives 20,000 independent replicates, for $28 \times 20{,}000 = 560{,}000$ complete population experiments.

For each combination, the output records the fixation count, point estimate, interval, exact prediction, seed and number of events used. The exact solution and the simulation are different ways to calculate the same model quantity. Their agreement is a check on our implementation.

Do not demand that all 28 nominal 95% intervals contain their exact predictions. Even correct sampling can produce an occasional miss. Instead, examine whether discrepancies have the scale expected from Monte Carlo error, and use deterministic tests of the equations to check mathematical correctness. A systematic pattern of large discrepancies deserves investigation; one slightly missed interval is not by itself proof of a bug.

The full study and the quick demonstration save their outputs separately. A quick run is useful for learning and checking that the code starts correctly. Its smaller sample size must not silently replace the full study in the guide's results.

**Question 1.** There are 37 fixations in 1,000 runs. What is the point estimate?
**Answer.** Divide fixations by runs: $\widehat{p} = 37/1000 = 0.037$, or 3.7%.

**Question 2.** You want to halve the standard error without changing the model. Roughly how should you change the number of runs?
**Answer.** Use four times as many. In equation (6.5), replacing $R$ with $4R$ multiplies the denominator inside the square root by four, dividing the standard error by two.

**Question 3.** Would one million repetitions prove that this model accurately describes a particular animal population?
**Answer.** No. More repetitions estimate the model's answer more precisely. They do not test whether its assumptions match that animal population. That requires biological evidence and possibly a different model.

# Chapter 7

# Derive the exact probability of fixation

## 7.1 An independent answer for checking the simulations

Repeated simulations estimate a probability. We now derive that probability directly from the model rules. The mathematical result is exact for this model. It is not an approximation obtained by averaging a finite number of computer runs.

Let $u_i$ denote the probability that A eventually fixes when the current A count is $i$. The subscript tells us the starting count for this question. For example, $u_1$ is the chance of eventual fixation from one A individual. It is not the chance that the *next event* increases the count.

The population size $N$ and A reproductive weight $r = 1+s$ stay fixed throughout this derivation. The transition probabilities $T_i^+$, $T_i^-$, and $T_i^0$ were derived in Chapter 3. They describe increase, decrease, and no change during a single event.

## 7.2 Start with the two answers we already know

If $i = 0$, no A individuals remain. With no mutation, A cannot return. Its probability of eventual fixation is therefore zero. If $i = N$, all individuals are already A, so fixation has already occurred and its probability is one. Thus

$$u_0 = 0, \qquad u_N = 1. \tag{7.1}$$

These are **boundary conditions**. They give the values of the unknown function at the two absorbing states. Our task is to find the values at the interior states $i = 1, 2, \ldots, N-1$.

## 7.3 Why the present count is enough

The next-event probabilities depend only on the current count and the fixed parameters. They do not depend on the sequence of earlier counts. A random process with this property has the **Markov property**.

Because the states are a finite list and events occur in discrete steps, we can call this process a **finite-state Markov chain**. That name does not add a new assumption. It names a property

of the rules already specified.

The Markov property lets us reuse the same fixation probabilities after one event. If the next count is $i+1$, the chance of eventual fixation from then onward is $u_{i+1}$. It does not matter how the process arrived at that count.

## 7.4 Condition on the first event

Start at an interior state $i$. Exactly one of three things happens next. With probability $T_i^+$, we move to $i+1$. With probability $T_i^-$, we move to $i-1$. With probability $T_i^0$, we stay at $i$.

For each possibility, multiply the probability of that first event by the probability of later fixation given its resulting state. Then add the three mutually exclusive possibilities:

$$u_i = T_i^+ u_{i+1} + T_i^- u_{i-1} + T_i^0 u_i. \tag{7.2}$$

This is called a **backward equation** or a **first-step recurrence**. We calculate eventual success from a given starting state by looking at the first step out of that state.

The term $T_i^0 u_i$ matters. A no-change event does not end the run. After that event, the process has the same count, so its remaining chance of fixation is still $u_i$.

For $N = 5$, $i = 2$, and $r = 1.5$, the one-event probabilities are 0.3, 0.2, and 0.5. The equation is

$$u_2 = 0.3u_3 + 0.2u_1 + 0.5u_2.$$

There is a thirty percent chance of moving to a state whose fixation probability is $u_3$, a twenty percent chance of moving to one whose fixation probability is $u_1$, and a fifty percent chance of remaining at the same state. None of these one-event probabilities is itself $u_2$.

## 7.5 Rearrange the recurrence without skipping steps

First subtract $T_i^0 u_i$ from both sides of equation (7.2):

$$(1 - T_i^0)u_i = T_i^+ u_{i+1} + T_i^- u_{i-1}.$$

Because the three transition probabilities add to one, $1 - T_i^0 = T_i^+ + T_i^-$. Substitute that expression:

$$(T_i^+ + T_i^-)u_i = T_i^+ u_{i+1} + T_i^- u_{i-1}.$$

Expand the left side, then move its two terms to the right:

$$\begin{aligned} 0 &= T_i^+ u_{i+1} - T_i^+ u_i + T_i^- u_{i-1} - T_i^- u_i \\ &= T_i^+ (u_{i+1} - u_i) - T_i^- (u_i - u_{i-1}). \end{aligned}$$

Finally move the second term to the other side:

$$T_i^+ (u_{i+1} - u_i) = T_i^- (u_i - u_{i-1}). \tag{7.3}$$

The equation now relates the improvement in fixation probability from one extra A individual to the improvement from the preceding extra individual.

## 7.6 Name the differences between neighbouring answers

Let $d_i$ denote the difference between the fixation probabilities at counts $i$ and $i-1$:

$$d_i = u_i - u_{i-1}, \qquad i = 1, 2, \ldots, N.$$

For example, $d_1 = u_1 - u_0 = u_1$, while $d_2 = u_2 - u_1$. Equation (7.3) becomes

$$T_i^+ d_{i+1} = T_i^- d_i.$$

For an interior count, both $i$ and $N-i$ are positive. The reproductive weight $r$ is positive too, so $T_i^+ > 0$. We may divide by it:

$$d_{i+1} = \frac{T_i^-}{T_i^+} d_i. \tag{7.4}$$

Now substitute the actual transition probabilities. Recall that $W_i = ri + N - i$ is their shared total reproductive weight:

$$\begin{aligned} \frac{T_i^-}{T_i^+} &= \frac{\dfrac{N-i}{W_i}\dfrac{i}{N}}{\dfrac{ri}{W_i}\dfrac{N-i}{N}} \\ &= \frac{(N-i)i}{ri(N-i)} \\ &= \frac{1}{r}. \end{aligned}$$

The factors $W_i$ and $N$ cancel first. The positive factors $i$ and $N-i$ cancel next. The important simplification is that the ratio does not depend on the current count:

$$d_{i+1} = \frac{d_i}{r}.$$

## 7.7 Follow the pattern of differences

Starting from $d_1$, the recurrence gives

$$d_2 = \frac{d_1}{r}, \qquad d_3 = \frac{d_2}{r} = \frac{d_1}{r^2}, \qquad d_4 = \frac{d_1}{r^3}.$$

Each new difference is the preceding difference multiplied by $1/r$. Such a pattern is a **geometric sequence**. For any allowed index $i$, the pattern is

$$d_i = d_1 r^{-(i-1)}.$$

A negative exponent means a reciprocal. For example, $r^{-2} = 1/r^2$.

To recover $u_i$, add the first $i$ differences. The intermediate terms cancel:

$$\begin{aligned} d_1 + d_2 + \cdots + d_i &= (u_1 - u_0) + (u_2 - u_1) + \cdots + (u_i - u_{i-1}) \\ &= u_i - u_0 \\ &= u_i. \end{aligned}$$

This cancellation is sometimes called a **telescoping sum**. In this case it just means that every intermediate $+u$ has an equal $-u$.

Let $j$ be an integer index used to count the powers in a sum. The symbol $\sum$ means "add the listed terms". We can now write

$$u_i = d_1 \sum_{j=0}^{i-1} r^{-j}.$$

For instance, when $i = 3$, the sum is $1 + r^{-1} + r^{-2}$. At the other boundary, $u_N = 1$, so

$$1 = d_1 \sum_{j=0}^{N-1} r^{-j}.$$

Dividing by the positive sum determines the unknown first difference:

$$d_1 = \frac{1}{\sum_{j=0}^{N-1} r^{-j}}.$$

Substitute this into the expression for $u_i$:

$$u_i = \frac{\sum_{j=0}^{i-1} r^{-j}}{\sum_{j=0}^{N-1} r^{-j}}. \tag{7.5}$$

## 7.8 Calculate the geometric sum explicitly

Assume for the moment that $r \neq 1$, meaning that $r$ is not equal to one. Let $q = 1/r$. Let $G_i$ denote the sum of the first $i$ powers beginning with $q^0 = 1$:

$$G_i = 1 + q + q^2 + \cdots + q^{i-1}.$$

Multiplying the whole sum by $q$ gives

$$qG_i = q + q^2 + \cdots + q^{i-1} + q^i.$$

Subtract the second equation from the first. The terms in the middle cancel:

$$G_i - qG_i = 1 - q^i.$$

Factor the left side to obtain $(1 - q)G_i = 1 - q^i$. Since $q \neq 1$, divide by $1 - q$:

$$G_i = \frac{1 - q^i}{1 - q}.$$

Apply this formula to both sums in equation (7.5). Their factors $1 - q$ cancel. Replacing $q$ by $1/r$ gives

$$\boxed{u_i = \frac{1 - r^{-i}}{1 - r^{-N}} \quad (r \neq 1).} \tag{7.6}$$

This is the exact Moran fixation probability under our stated rules. Its single-initial-copy form is a standard result discussed, with a log-fitness convention, by McCandlish and colleagues [1].

For one initial A individual, substitute $i = 1$:

$$u_1 = \frac{1 - r^{-1}}{1 - r^{-N}}.$$

For any starting count, substitute $r = 1 + s$ to express the same answer using our advantage parameter $s$.

## 7.9 Handle neutrality separately

If $s = 0$, then $r = 1$. Substituting $r = 1$ directly into equation (7.6) gives zero divided by zero, which is undefined. That does not mean the model has no answer. It means that this particular shortened expression needs a separate neutral case.

Return to equation (7.5). When $r = 1$, every power $r^{-j}$ equals one. The numerator contains $i$ ones. The denominator contains $N$ ones. Therefore

$$\boxed{u_i = \frac{i}{N} \quad (s = 0).} \tag{7.7}$$

For one initial A individual, the neutral probability is $1/N$. The neutral fixation probability equals the initial frequency, not one half unless the initial frequency happens to be one half.

## 7.10 A complete five-individual calculation

Take $N = 5$ and $s = 0.5$, so $r = 1.5 = 3/2$ and $r^{-1} = 2/3$. First compute the denominator of equation (7.6):

$$1 - r^{-N} = 1 - \left(\frac{2}{3}\right)^5 = 1 - \frac{32}{243} = \frac{211}{243}.$$

Starting from two A individuals, the numerator is

$$1 - r^{-2} = 1 - \left(\frac{2}{3}\right)^2 = 1 - \frac{4}{9} = \frac{5}{9}.$$

Dividing by a fraction means multiplying by its reciprocal:

$$u_2 = \frac{5/9}{211/243} = \frac{5}{9}\frac{243}{211} = \frac{135}{211} \approx 0.6398.$$

The symbol $\approx$ means "approximately equal to". The exact fraction is $135/211$; the decimal is rounded.

Starting from just one A individual, the numerator is $1 - 2/3 = 1/3$, so

$$u_1 = \frac{1/3}{211/243} = \frac{81}{211} \approx 0.3839.$$

Even this substantial reproductive advantage does not make fixation certain. With one initial copy, the loss probability is $1 - u_1 = 130/211 \approx 0.6161$.

For comparison, the neutral probabilities are $2/5 = 0.4$ from two copies and $1/5 = 0.2$ from one copy. Selection improves the odds in both comparisons. The absolute probability and the improvement over neutrality are different quantities.

## 7.11 Check the answer using simultaneous equations

There is another way to solve the backward equations. For a small population, we can solve the interior equations together without using the geometric-sum formula. This gives an independent check on the derivation and the code.

Take $N = 3$ and $r = 2$, so $s = 1$. This larger advantage is used only to make the arithmetic short. The unknown probabilities are $u_1$ and $u_2$; the boundaries are $u_0 = 0$ and $u_3 = 1$.

At $i = 1$, the total weight is $2(1) + 2 = 4$. The probabilities are $T_1^+ = 1/3$, $T_1^- = 1/6$, and $T_1^0 = 1/2$. The backward equation is

$$u_1 = \frac{1}{3}u_2 + \frac{1}{6}(0) + \frac{1}{2}u_1.$$

Subtract $u_1/2$, then multiply by six, to get

$$3u_1 = 2u_2, \qquad \text{so} \qquad u_1 = \frac{2}{3}u_2.$$

At $i = 2$, the total weight is $2(2) + 1 = 5$. The probabilities are $T_2^+ = 4/15$, $T_2^- = 2/15$, and $T_2^0 = 9/15$. Hence

$$u_2 = \frac{4}{15}(1) + \frac{2}{15}u_1 + \frac{9}{15}u_2.$$

Multiply by fifteen and collect the $u_2$ terms:

$$15u_2 = 4 + 2u_1 + 9u_2, \qquad \text{so} \qquad 6u_2 = 4 + 2u_1.$$

Substitute $u_1 = 2u_2/3$, then multiply by three:

$$\begin{aligned} 6u_2 &= 4 + \frac{4}{3}u_2, \\ 18u_2 &= 12 + 4u_2, \\ 14u_2 &= 12. \end{aligned}$$

Thus $u_2 = 6/7$, and $u_1 = (2/3)(6/7) = 4/7$.

The exact formula gives the same results:

$$u_1 = \frac{1 - 1/2}{1 - 1/8} = \frac{4}{7}, \qquad u_2 = \frac{1 - 1/4}{1 - 1/8} = \frac{6}{7}.$$

For larger populations, we ask Julia to solve all $N - 1$ interior equations together. A matrix is a rectangular arrangement of numbers that can store the coefficients of such equations. The numerical solver handles the elimination. We still supply the equations and the two boundary values. Agreement with the closed formula is a useful check because the numerical procedure does not evaluate that formula to obtain its answer.

## 7.12 A preview of stable numerical evaluation

Mathematically, equation (7.6) is correct. A computer, however, stores numbers with a finite number of significant digits. When $s$ is extremely small, both numerator and denominator subtract two nearly equal numbers. Some useful digits can be lost.

The natural logarithm, written log, and the exponential function, written exp, let us rewrite the powers. They are inverse functions: $\exp(\log r) = r$ for positive $r$. Let $\sigma = \log r = \log(1 + s)$. This Greek letter is read as "sigma" and is the difference in log fitness used by our reference paper. It is not exactly the same as $s$.

Using $r^{-i} = \exp(-i\sigma)$, the same probability is

$$u_i = \frac{1 - \exp(-i\sigma)}{1 - \exp(-N\sigma)}.$$

Julia provides `log1p(s)` to calculate $\log(1+s)$ accurately when $s$ is small. It provides `expm1(z)` to calculate $\exp(z) - 1$ accurately when its argument, called $z$ here, is near zero. We can use `expm1` in both numerator and denominator. Both signs reverse, so the ratio is unchanged. The neutral case still needs its explicit answer $i/N$.

These functions change numerical accuracy, not the biological model. The code walkthrough explains their use after first showing the readable formula.

**Question 1.** A neutral population has $N = 20$ and starts with $i = 3$. What is the fixation probability?
**Answer.** Neutrality gives $u_i = i/N$, so $u_3 = 3/20 = 0.15$. The loss probability is $1-0.15 = 0.85$.
**Question 2.** In the three-individual example, verify the first backward equation using $u_1 = 4/7$ and $u_2 = 6/7$.
**Answer.** Its right side is $(1/3)(6/7) + (1/6)(0) + (1/2)(4/7) = 2/7 + 0 + 2/7 = 4/7$, which equals its left side $u_1$.
**Question 3.** Why is it wrong to identify $T_1^+$ with $u_1$?
**Answer.** The first quantity is the probability of an increase in the next single event. The second is the probability of eventually reaching the all-A boundary, possibly after many increases, decreases, and unchanged events. An initial increase can still be followed by loss.

## 7.13 What we have established

The exact answer came from four ingredients: the event probabilities, the Markov property, conditioning on the first event, and the two absorbing boundaries. No simulation output was used to derive it. We can therefore use it as an external target when assessing simulation estimates. Differences of the expected sampling size are normal. Large systematic discrepancies would instead suggest a mismatch in assumptions, a numerical problem, or an implementation error.

# Chapter 8

# What an average-change model can and cannot predict

## 8.1 Two useful questions, two different calculations

The stochastic model asks what can happen when every birth and replacement is random. "Stochastic" means that randomness is part of the description. A *deterministic* model instead gives one prescribed path from a given starting point. It has no random draws.

A deterministic path can describe a systematic tendency, such as an advantageous type tending to increase. It cannot display all the individual histories in which that type is lost. We will derive a deterministic approximation from our stochastic rules, rather than introduce an unrelated growth equation.

This chapter also develops a simpler approximation to the fixation probability. Keep the two approximations separate:

- The deterministic approximation concerns a population's changing allele frequency.
- The weak-selection approximation concerns the probability of the eventual fixation outcome.

They answer different questions.

## 8.2 First calculate the expected change in the allele count

Let $I_j$ be the random number of focal-allele carriers after $j$ reproduction events. The capital letter reminds us that different simulation runs can have different counts. Suppose its present value is $i$.

During one event, the count increases by one with probability $T_i^+$, decreases by one with probability $T_i^-$, or stays unchanged with probability $T_i^0$. Expected change means each possible change multiplied by its probability, then added:

$$\mathbb{E}[I_{j+1} - I_j \mid I_j = i] = (+1)T_i^+ + (-1)T_i^- + 0T_i^0 \tag{8.1}$$

$$= T_i^+ - T_i^-. \tag{8.2}$$

The vertical bar means "given that". We are calculating the expectation given the current count, not taking an average over all possible current counts.

Write the focal type's relative fitness as $r = 1 + s$. Substituting the transition probabilities gives

$$T_i^+ - T_i^- = \frac{ri}{ri + N - i}\frac{N - i}{N} - \frac{N - i}{ri + N - i}\frac{i}{N} \tag{8.3}$$

$$= \frac{ri(N - i) - i(N - i)}{N(ri + N - i)} \tag{8.4}$$

$$= \frac{(r - 1)i(N - i)}{N(ri + N - i)} \tag{8.5}$$

$$= \frac{s\, i(N - i)}{N(N + si)}. \tag{8.6}$$

The last line used $r - 1 = s$ and $ri + N - i = (1 + s)i + N - i = N + si$.

> Return to $N = 5$, $i = 2$ and $s = 0.5$. We already found $T_i^+ = 0.3$ and $T_i^- = 0.2$. The expected count change is $0.3 - 0.2 = 0.1$.
> No actual event increases the count by one tenth of an individual. Actual changes are $+1$, $-1$ or $0$. The number 0.1 is their probability-weighted average across many populations that all currently have two carriers.

## 8.3 Convert a count change into a frequency change

The allele frequency is the carrier count divided by population size. Define the random frequency $X_j = I_j/N$, and write $x = i/N$ for a particular current frequency. A count change of one changes the frequency by $1/N$.

Divide equation (8.6) by $N$, then substitute $i = Nx$ and $N - i = N(1 - x)$:

$$\mathbb{E}[X_{j+1} - X_j \mid X_j = x] = \frac{s\, i(N - i)}{N^2(N + si)} \tag{8.7}$$

$$= \frac{s\, N^2 x(1 - x)}{N^3(1 + sx)} \tag{8.8}$$

$$= \frac{s\, x(1 - x)}{N(1 + sx)}. \tag{8.9}$$

In the five-individual example, the frequency is $x = 2/5 = 0.4$. The expected count change 0.1 corresponds to an expected frequency change $0.1/5 = 0.02$.

## 8.4 Turn that expected change into one deterministic path

Let $m_j$ be a deterministic approximation to the frequency after event $j$. We allow $m_j$ to be any real number between zero and one. It need not be a multiple of $1/N$, because it describes an approximation, not the literal count of individuals.

Start it at the actual initial frequency, $m_0 = i_0/N$, where $i_0$ is the initial carrier count. Then repeatedly add the expected frequency change evaluated at the current deterministic frequency:

$$m_{j+1} = m_j + \frac{s\, m_j(1 - m_j)}{N(1 + sm_j)}. \tag{8.10}$$

A rule that calculates the next value from the current value is called a *recurrence*. Here the recurrence is nonlinear: it contains products involving $m_j$ and also $m_j$ in a denominator.

Take $N = 5$, $i_0 = 2$ and $s = 0.5$. The initial value is $m_0 = 0.4$. The first update is

$$m_1 = 0.4 + \frac{0.5 \times 0.4 \times 0.6}{5(1 + 0.5 \times 0.4)} = 0.4 + 0.02 = 0.42.$$

For the second update we use 0.42, not the old value 0.4:

$$m_2 = 0.42 + \frac{0.5 \times 0.42 \times 0.58}{5(1 + 0.5 \times 0.42)} \approx 0.440132.$$

These values trace one smooth tendency. An actual population of five individuals cannot have frequency 0.42: its possible frequencies are 0, 0.2, 0.4, 0.6, 0.8 and 1.

The code needs only a loop for this recurrence: read the previous value, calculate the increment, and store their sum. It contains no random draw. The deterministic and stochastic functions therefore need different explanations even when they appear on the same figure.

## 8.5 Why this is not usually the exact average of the stochastic runs

It is tempting to call $m_j$ the exact average frequency. That is generally incorrect.

Define the function

$$g(x) = \frac{s\,x(1-x)}{N(1+sx)}.$$

The stochastic model's exact relation is

$$\mathbb{E}[X_{j+1}] = \mathbb{E}[X_j] + \mathbb{E}[g(X_j)].$$

To obtain our deterministic recurrence, we replace the average of $g(X_j)$ with $g$ evaluated at an average-like frequency. In general,

$$\mathbb{E}[g(X_j)] \neq g(\mathbb{E}[X_j]).$$

Here is a simpler example showing why those operations do not commute. Suppose a number is zero half the time and one half the time. Its average is $1/2$. Its average square is $(0^2+1^2)/2 = 1/2$. But the square of its average is $(1/2)^2 = 1/4$. Average first, then square, gives a different result from square first, then average.

Our drift function is also nonlinear. Treating its value at a typical frequency as the average drift is called a *mean-field approximation*. This is a useful approximation, but it needs that label.

There is an instructive special case. Under neutrality, $s = 0$, so $g(x) = 0$ at every frequency. The exact ensemble-average frequency then remains at its initial value. Individual populations can still drift to zero or one. A flat average does not imply unchanging individual populations.

## 8.6 Time units and the differential equation

So far, one time step has meant one birth–replacement event. A population with 200 individuals experiences less change in frequency from one such event than a population with five individuals.

For comparison we can define a rescaled time

$$t = \frac{j}{N}.$$

Thus one unit of $t$ contains $N$ events. We may describe this as a generation-like unit, but it is *not* a biological generation with synchronous parents and offspring. Our Moran population has overlapping replacement events. Always inspect the axis label before interpreting a trajectory.

The slope of a curve means change in height divided by change in horizontal position. For frequency over rescaled time, this is

$$\frac{\text{frequency change}}{\text{time change}}.$$

One event changes rescaled time by $1/N$. Dividing the expected frequency increment from equation (8.9) by $1/N$ cancels the factor $N$:

$$\frac{sx(1-x)/(N(1+sx))}{1/N} = \frac{sx(1-x)}{1+sx}.$$

A *derivative*, written $dx/dt$, is the limiting slope over an increasingly small time interval. When a continuous curve is an appropriate approximation to the discrete updates, its equation is

$$\frac{dx}{dt} = \frac{s\,x(1-x)}{1+sx}. \tag{8.11}$$

An equation involving a derivative is called a *differential equation*. We do not need a specialised differential-equation solver to generate the deterministic curves in this project: the implemented recurrence already supplies the values at event times. The differential equation explains the continuous-time interpretation of that recurrence.

For $s > 0$ and $0 < x < 1$, every factor in the right-hand side is positive. The deterministic curve therefore rises towards one. At $x = 0$ or $x = 1$, the factor $x(1-x)$ is zero, so these endpoints do not move. For $s = 0$, the entire right-hand side is zero, so the deterministic frequency stays flat.

**Question.** Does a positive deterministic slope prove that the focal allele fixes in every stochastic run?
**Worked answer.** No. A positive slope describes a systematic tendency. The stochastic model still permits downward events. A rare advantageous allele can disappear before that tendency produces a large population of carriers. The exact fixation probability answers the eventual-outcome question.

## 8.7 An advantageous allele can have better odds and still usually disappear

For one initial copy, the exact fixation probability is

$$p_N(s) = \frac{1-(1+s)^{-1}}{1-(1+s)^{-N}}, \qquad s > 0. \tag{8.12}$$

The subscript $N$ reminds us that population size matters. Under neutrality, $p_N(0) = 1/N$.

For $N = 100$ and $s = 0.01$, the exact selected probability is approximately 0.015709. The neutral probability is 0.01. Selection therefore increases fixation probability by a factor of about

$$\frac{0.015709}{0.01} \approx 1.571.$$

That is about a 57% relative increase. Yet the absolute fixation chance is only about 1.57%. The corresponding loss chance is about 98.43%.

Both statements are true. Selection improves the odds appreciably relative to neutrality, while loss remains by far the more common outcome for one rare copy. Keeping absolute probability separate from relative improvement is a central lesson of this study.

## 8.8 Build the weak-selection approximation step by step

The exact formula is already easy to evaluate. Why approximate it? A simpler expression can reveal which combinations of parameters matter. We must then check the price paid in accuracy.

First simplify the numerator exactly:

$$1 - (1 + s)^{-1} = 1 - \frac{1}{1 + s} = \frac{1 + s - 1}{1 + s} = \frac{s}{1 + s}.$$

When $s$ is small compared with one, the denominator $1+s$ is close to one. Therefore $s/(1+s) \approx s$. The symbol $\approx$ means "approximately equal", rather than exactly equal.

Next rewrite the denominator using exponential and logarithmic functions. The function log is the natural logarithm, which reverses the exponential function exp. The identity $a^b = \exp(b \log a)$ gives

$$(1 + s)^{-N} = \exp[-N \log(1 + s)].$$

Near zero, $\log(1 + s)$ is approximately $s$. The first terms of its expansion are

$$\log(1 + s) = s - \frac{s^2}{2} + \frac{s^3}{3} - \cdots .$$

The dots indicate further, higher-power terms. For small $s$, those higher powers are small, which motivates retaining only the first term. This gives

$$(1 + s)^{-N} \approx \exp(-Ns).$$

Applying both approximations yields

$$q_N(s) = \frac{s}{1 - \exp(-Ns)}. \tag{8.13}$$

Here $q_N(s)$ names our approximate probability. It approximates $p_N(s)$ when selection is weak. Weak selection means small $s$, not necessarily small $Ns$.

Our reference uses a selection variable equal to the difference in log fitnesses. Call that variable $\sigma$. In our convention,

$$\sigma = \log(1 + s).$$

Its exact formula can then be written $(1 - \exp(-\sigma))/(1 - \exp(-N\sigma))$. The two conventions agree after this conversion, but $s$ and $\sigma$ are not exactly the same number [1]. Likewise, fixation approximations from a different reproduction model must not be transferred by copying a factor such as $2s$ without checking that model's conventions.

## 8.9 Understand the small and large parameter limits

At $s = 0$, equation (8.13) appears to divide zero by zero. We assign its limiting value $1/N$, consistent with the exact neutral result. To see why, use $\exp(-Ns) \approx 1 - Ns$ when $Ns$ is small:

$$q_N(s) \approx \frac{s}{1-(1-Ns)} = \frac{s}{Ns} = \frac{1}{N}.$$

Close to neutrality, the initial one-copy frequency remains the leading fixation prediction.

If $Ns$ is large and positive, $\exp(-Ns)$ is close to zero. The approximation instead becomes $q_N(s) \approx s$. This conclusion still requires $s$ itself to be small if we want it to be accurate. "Large $Ns$" does not cancel the weak-selection assumption.

For the exact formula, keep $s > 0$ fixed and increase $N$. Then $(1+s)^{-N}$ tends to zero, so

$$p_N(s) \longrightarrow \frac{s}{1+s}. \tag{8.14}$$

The arrow means "approaches as $N$ grows". The exact limit differs from the approximate limit $s$. When $s$ is tiny, their difference is tiny. When $s = 0.5$, the exact limit is $1/3$, while the weak-selection expression approaches $1/2$. That is a substantial error.

## 8.10 Measure approximation error rather than guessing

Define relative error as

$$\text{relative error} = \frac{|q_N(s) - p_N(s)|}{p_N(s)}. \tag{8.15}$$

The absolute-value bars remove the sign of the difference. Dividing by the exact probability expresses the discrepancy as a fraction of that probability. Multiplying by 100 gives a percentage.

For $N = 100$ and $s = 0.01$, the exact probability is approximately 0.0157087 and the approximation is approximately 0.0158198. Their relative error is approximately 0.00707, or 0.707%.

For $N = 100$ and $s = 0.5$, the exact probability is essentially $1/3$ and the approximation is essentially $1/2$. The relative error is about 50%. This larger coefficient is included in an analytical error grid to expose the approximation's limitations. It is not a measured biological fitness estimate, and it is not part of the main simulation grid.

The error curves use direct mathematical evaluations, so they do not contain Monte Carlo sampling error. This separates an approximation's mathematical error from the simulation's finite-sample uncertainty.

## 8.11 What the product $Ns$ does and does not mean

The weak-selection expression contains the product $Ns$. Divide its approximate fixation probability by the neutral probability $1/N$:

$$\frac{q_N(s)}{1/N} = N q_N(s) = \frac{Ns}{1-\exp(-Ns)}.$$

This relative advantage depends only on $Ns$. It explains why the product is a useful guide to the balance between selection and drift in this setting. It does not make $Ns$ an exact universal description of all population-genetic models.

Even within our model, the exact relative probabilities are only approximately collapsed by $Ns$. For example, the parameter pairs $(N, s) = (20, 0.05)$, $(100, 0.01)$ and $(200, 0.005)$ all give $Ns = 1$. Their exact values of $Np_N(s)$ are approximately 1.5284, 1.5709 and 1.5764. They move closer to the weak-selection prediction as $s$ becomes smaller.

## 8.12 One initial copy is not the same as one initial frequency

Changing $N$ while keeping one initial copy changes the initial frequency. One copy means frequency $1/20 = 0.05$ in a population of 20, but $1/200 = 0.005$ in a population of 200. The second allele starts ten times rarer as a fraction of its population.

At fixed $s > 0$, the one-copy exact probability actually decreases towards $s/(1+s)$ as $N$ increases: the numerator stays fixed while the denominator increases towards one. For example, with $s = 0.1$, the exact probabilities at $N = 100$ and $N = 200$ are about 0.0909157 and 0.0909091. Larger $N$ does not automatically improve the absolute fixation chance of one initial copy.

Now hold the initial frequency at 10% instead. We then start with $i_0 = N/10$ copies: two copies for $N = 20$, five for $N = 50$, ten for $N = 100$ and twenty for $N = 200$. All four populations begin with the same fraction of carriers. Under neutrality, all four have fixation probability 0.1. Their initial counts differ, but their initial frequencies do not.

Our supplementary analytical comparison makes this distinction visible. It uses the general exact formula with the appropriate $i_0$, rather than pretending that a one-copy comparison controls initial frequency. For instance, at $N = 100$ and $s = 0.01$, ten initial copies give a fixation probability of about 0.15027, whereas one initial copy gives about 0.01571.

**Question 1.** Set $s = 0$ in the deterministic recurrence. What happens? Does the stochastic population do the same thing?
**Answer.** The deterministic increment becomes zero, so its frequency stays at $i_0/N$. The stochastic population still experiences upward, downward and unchanged events. It can eventually lose or fix the allele, even though its expected frequency remains unchanged.

**Question 2.** A calculated exact probability is 0.02 and an approximation gives 0.021. What is the relative error?
**Answer.** The absolute difference is $|0.021 - 0.02| = 0.001$. Divide by the exact value: $0.001/0.02 = 0.05$, or 5%.

**Question 3.** In a population of 200, how many copies correspond to an initial frequency of 10%?
**Answer.** Convert 10% to 0.1, then multiply by the population size: $0.1 \times 200 = 20$ copies. Starting with one copy would instead mean $1/200 = 0.005$, or 0.5%.

**Question 4.** If a weak-selection approximation is inaccurate at $s = 0.5$, should we change the simulation to make it agree with that approximation?
**Answer.** No. Check the simulation against the exact model solution. An approximation can fail because its assumptions are unsuitable. That failure is a result to explain, not a reason to alter correct simulation rules.

# Chapter 9

# Read the results, one figure at a time

## 9.1 What was actually run?

The full study used four population sizes: $20, 50, 100, 200$. For each size, it used seven selective advantages: $0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1$. Four multiplied by seven gives 28 conditions. Each condition started with one focal individual and was repeated 20,000 times. That gives 560000 completed fixation-or-loss experiments.

Each experiment followed the same model rules but used fresh pseudorandom draws. It continued until the focal allele was either lost or fixed. We did not stop it early and classify an unfinished experiment as a loss. Separate files store a small number of histories for drawing. Those displayed histories have a fixed time horizon; they are not the experiment used to estimate fixation probability.

The exact probability and the independent recurrence check were calculated without sampling. The approximation comparison is also analytical: it compares two mathematical expressions, not two sets of random experiments. Keeping these three kinds of output separate makes the study easier to interpret.

## 9.2 Figure 1: how can a growing deterministic curve coexist with loss?

Figure 9.1 shows eight preselected histories in grey for each condition. The seeds were set before looking at the outcomes. The dashed teal curve is the deterministic comparison. It is not a ninth random history and is not generally the exact ensemble average.

Begin with the upper-left panel. The allele starts in one of 100 individuals, so its frequency begins at 0.01. There is no selection. The deterministic comparison stays at 0.01 because its expected change is zero. The grey paths can still move: zero expected change does not mean zero random variation. Most single-copy neutral alleles are lost.

Next compare the left and right columns. Ten initial copies mean a frequency of 0.1, not a stronger selective advantage. There are more initial copies to lose. These are different starting conditions, so their outcomes need not resemble each other.

Finally compare rows. A positive $s$ makes the deterministic curve increase. A stochastic history can nevertheless hit zero. Once it reaches zero, the mutation-free model has no mechanism to

restore the allele. A later expected advantage cannot reproduce an allele that is absent.

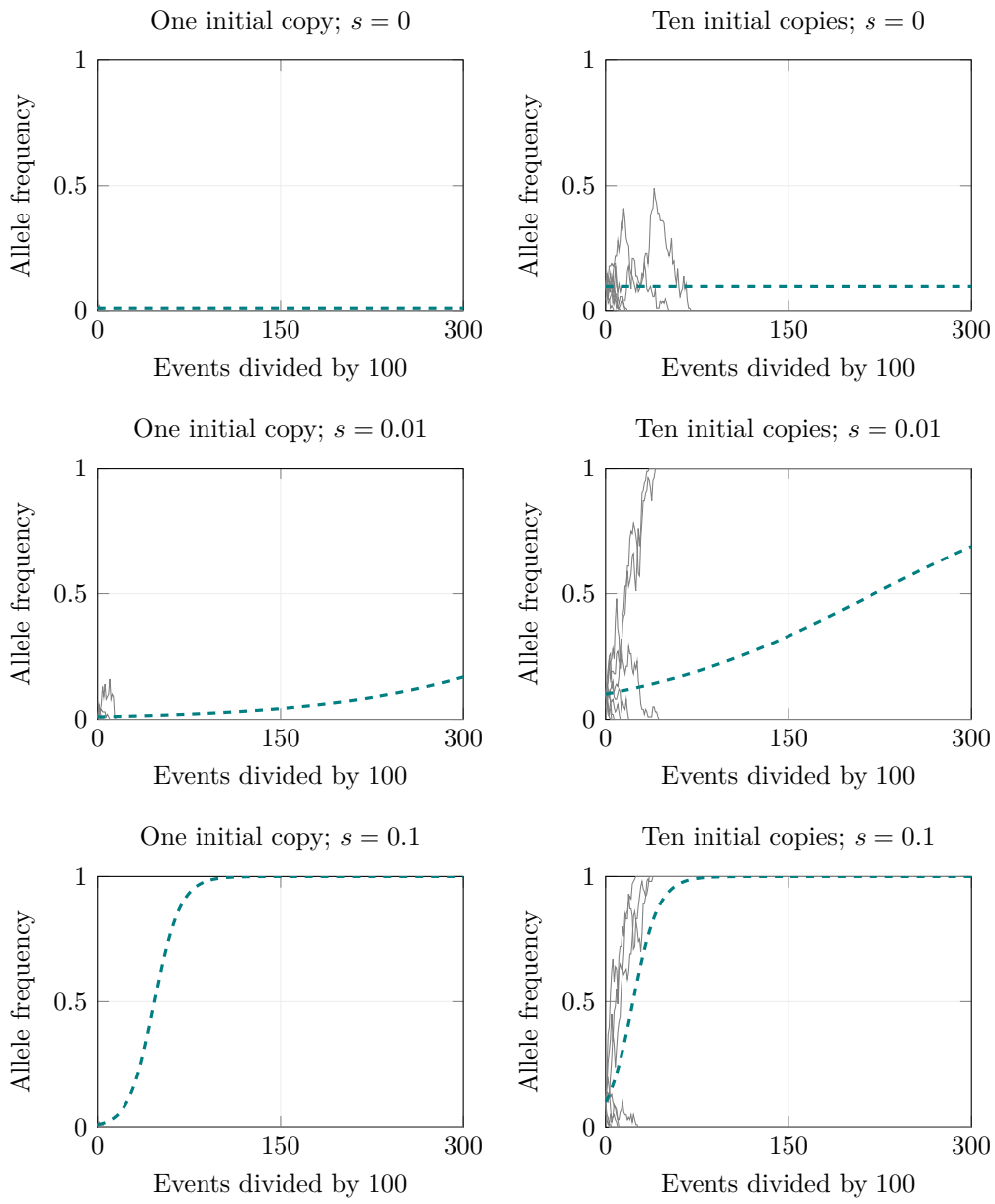



Figure 9.1: Eight preselected stochastic histories per panel (grey) and a deterministic mean-field comparison (teal dashed). Every panel has $N = 100$. Rows change selective advantage; columns change the initial number of copies. The horizontal coordinate is event count divided by 100, a conventional generation-like time unit. The simulation updates every event; the CSV records every 100 events for display. Histories that reach a boundary remain there. These few histories illustrate possible behaviour; the separate 20,000-repetition experiments estimate probabilities.

**Read the axes before reading a path.** The horizontal axis measures elapsed model events divided by population size. An x-coordinate of ten means 1,000 events here. It does not mean ten discrete nonoverlapping generations. The vertical axis is a fraction, from zero to one. Multiplying it by 100 gives the focal count for this figure.

Do not estimate a rare probability from eight paths. If an outcome has probability 0.01, eight independent runs usually contain no such outcome. That would be entirely compatible with the exact probability. The larger experiment is needed for estimating rare outcomes.

## 9.3 Figure 2: do the simulations agree with the exact calculation?

Figure 9.2 places selective advantage on the horizontal axis and eventual fixation probability on the vertical axis. Each panel has a different population size. All start with one copy. A teal line gives exact probabilities. A dot gives the observed proportion of fixations in 20,000 repetitions. A vertical bar gives that estimate's 95% Wilson interval.

At $s = 0$, the exact answer is $1/N$. The neutral intercept therefore differs between panels: 0.05, 0.02, 0.01, and 0.005. The panels share the same vertical scale so that these differences are visible.

Within a fixed population size, the exact line increases with advantage. The dots need not increase at every adjacent setting. Each dot comes from a separate finite experiment, so sampling noise can obscure a small difference. A local downward movement between two estimated points is not, by itself, evidence that a beneficial allele became less likely to fix.

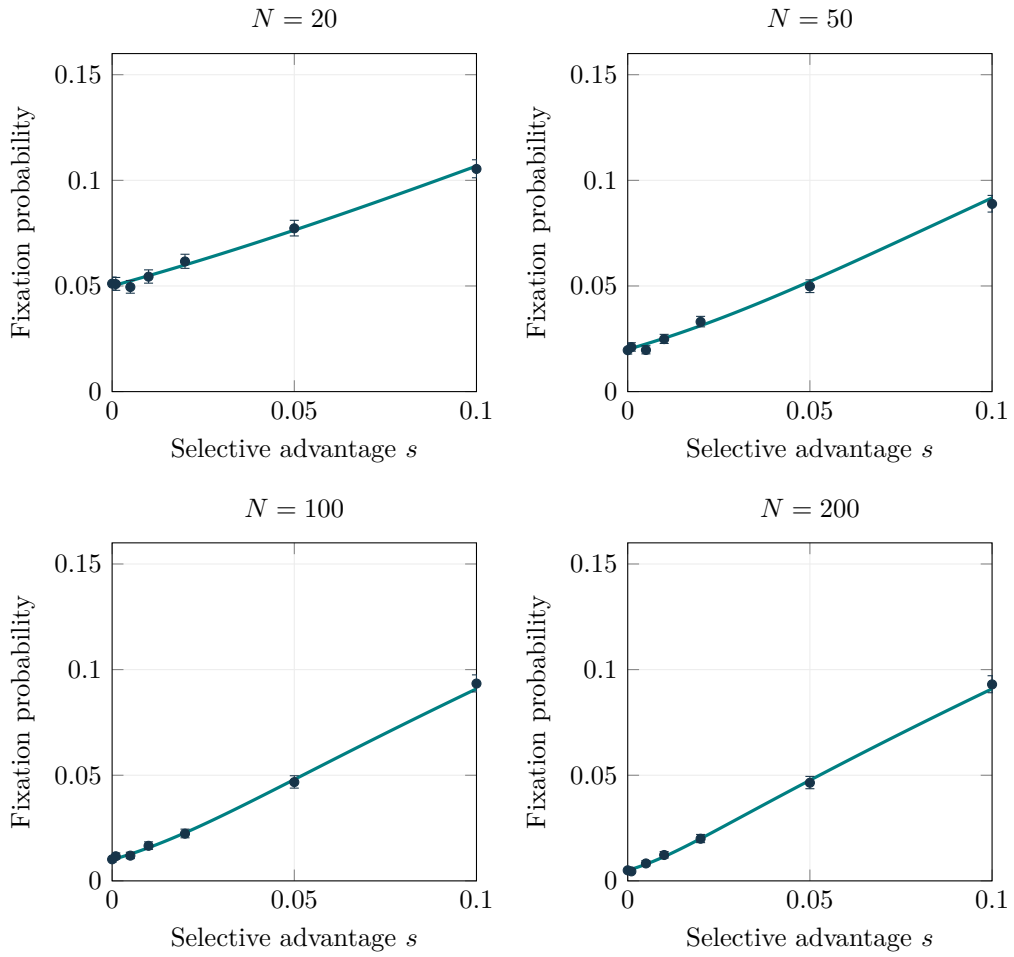



Figure 9.2: Exact fixation probabilities (teal lines), simulation estimates (dots), and two-sided 95% Wilson intervals (bars). Each point uses 20,000 completed experiments and one initial copy. Differences from the exact line are expected sampling variation; confidence intervals are not intended to contain the exact answer in every one of many independent experiments.

### 9.3.1 Read one result all the way through

For $N = 100, s = 0.01$, the supplied full run produced 332 fixations in 20,000 repetitions. Divide to get the estimated probability:

$$\widehat{p} = \frac{332}{20000} = 0.0166 = 1.66\%.$$

The exact model probability is approximately 0.0157087, or 1.57087%. Its predicted number of fixations in 20,000 repetitions is $20000 \times 0.0157087$, approximately 314.17. An expected count need not be a whole number because it is an average across hypothetical repetitions of the entire batch.

The actual count 332 is about 18 above that expected count. The exact binomial standard error of the probability estimate is

$$\sqrt{\frac{0.0157087(1 - 0.0157087)}{20000}} \approx 0.000879.$$

The discrepancy $0.0166 - 0.0157087 \approx 0.000891$ is about one such standard error. That is an ordinary size of sampling difference. The Wilson interval is approximately $[0.01492, 0.01847]$; it includes the exact probability in this case.

**Better odds are not the same as likely success.** The neutral probability at $N = 100$ is 0.01. Dividing 0.0157087 by 0.01 gives approximately 1.57. Selection increased the fixation chance by a factor of about 1.57. But the loss probability remains $1 - 0.0157087 \approx 0.98429$, or 98.43%. Both statements describe the same result.

### 9.3.2 The complete set of settings for one population size

Table 9.1 is imported directly from the generated data. Every row here has $N = 100$, one initial copy, and 20,000 repetitions. Probabilities in the table are fractions, not percentages. Multiply by 100 to obtain percentages.

| $s$ | Fixations | Estimate | Exact | 95% Wilson interval |
|---|---|---|---|---|
| 0.000 | 204 | 0.0102 | 0.0100 | [0.0089, 0.0117] |
| 0.001 | 233 | 0.0117 | 0.0105 | [0.0103, 0.0132] |
| 0.005 | 239 | 0.0120 | 0.0127 | [0.0105, 0.0136] |
| 0.010 | 332 | 0.0166 | 0.0157 | [0.0149, 0.0185] |
| 0.020 | 447 | 0.0223 | 0.0227 | [0.0204, 0.0245] |
| 0.050 | 935 | 0.0467 | 0.0480 | [0.0439, 0.0498] |
| 0.100 | 1868 | 0.0934 | 0.0909 | [0.0894, 0.0975] |

Table 9.1: The supplied full run for $N = 100$. Rounding in the printed table does not affect the calculations, which retain more digits in the CSV.

### 9.3.3 One interval misses: should we discard that experiment?

Across all 28 conditions, 27 intervals contain their exact model probabilities. The interval for $N = 50, s = 0.005$ misses: the estimate is 0.0197, while the exact value is about 0.022541. The largest discrepancy anywhere in the grid is 2.707 exact binomial standard errors.

This does not mean that we should choose a new seed until all intervals cover. Repeated 95% intervals are not supposed to cover in every experiment. Selecting a seed because its results look especially close would hide the variation we are trying to understand. We retain the specified seeds and report the miss. The deterministic recurrence check and the event-rule checks independently support the correctness of the calculations.

## 9.4 Do larger populations give a new single copy a better chance?

There are two different comparisons. One holds the initial count at one. The other holds the initial *frequency* at 10%. With $N = 20$, 10% means two copies. With $N = 200$, it means 20 copies. Changing population size while holding frequency fixed also changes the initial number of copies.

Table 9.2 uses $s = 0.01$ and exact calculations, not additional Monte Carlo estimates. Its final column divides the single-copy probability by its neutral baseline $1/N$.

| $N$ | One copy | Initial frequency 10% | One copy / neutral |
|---|---|---|---|
| 20 | 0.0549 | 0.1092 | 1.097 |
| 50 | 0.0253 | 0.1238 | 1.263 |
| 100 | 0.0157 | 0.1503 | 1.571 |
| 200 | 0.0115 | 0.2090 | 2.294 |

Table 9.2: Exact fixation probabilities under two starting rules, at $s = 0.01$. The last column is a dimensionless ratio, not a probability.

For a single copy, increasing $N$ decreases the starting frequency. At fixed positive advantage, its exact fixation probability decreases towards $s/(1 + s)$. Meanwhile the neutral baseline falls as $1/N$, so the *ratio to neutral* can increase. There is no contradiction: an absolute probability and a ratio to a changing baseline answer different questions.

## 9.5 Figure 3: how wrong is a useful approximation?

Figure 9.3 compares the weak-selection expression

$$q = \frac{s}{1 - e^{-Ns}}$$

with the exact single-copy probability $p$. Its vertical quantity is $100|q-p|/p$, the absolute relative error expressed as a percentage. This is *not* the fixation probability. For example, a 10% relative error means that the approximation differs by one tenth of the exact probability; it does not necessarily differ by ten percentage points.

The horizontal axis is logarithmic. Equal spacing represents equal ratios rather than equal differences. Moving from $10^{-5}$ to $10^{-4}$ multiplies advantage by ten, just as moving from $10^{-3}$ to $10^{-2}$ does. Chapter 8 explains the powers and logarithms used here.

At small advantage, the approximation is close. At larger advantage, its error becomes substantial. A large value of $Ns$ does not by itself license replacing $\log(1+s)$ with $s$. Those are different assumptions. We can use the exact formula throughout this project, so approximation error is something to investigate rather than an unavoidable source of uncertainty.

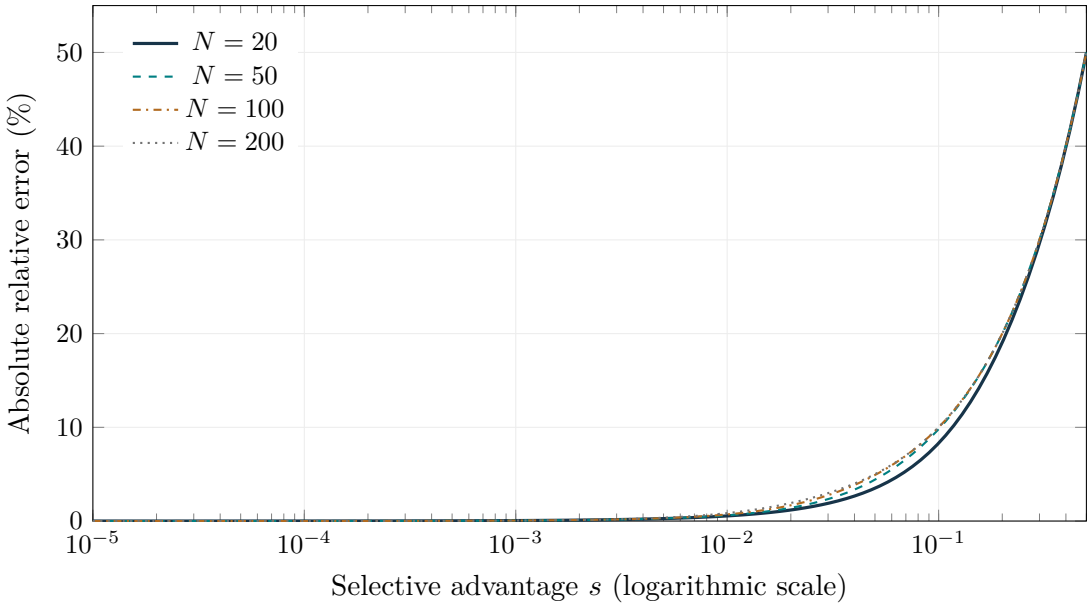



Figure 9.3: Error in the weak-selection approximation for one initial copy. Lines compare analytical expressions for four population sizes. The advantage range extends to 0.5 specifically to examine failure of the small-advantage assumption. These settings are illustrative mathematical inputs, not fitted biological parameters.

## 9.6 What have we learned, and what have we not learned?

We learned that reproductive advantage increases fixation odds at a given population size under these rules, yet loss can remain the overwhelmingly likely outcome for a single copy. We learned how to calculate that probability exactly, estimate it by simulation, quantify sampling uncertainty, and check a simpler approximation.

We did not infer an advantage from measured genetic data. We did not include diploidy, dominance, mutation, migration, population structure, changing population size, or a fluctuating environment. A successful check says that our implementation agrees with the stated model. Applying the model to a particular species would require checking whether its assumptions are suitable and how its parameters relate to observations.

**Try it.** Why does one exact curve plus a cloud of simulation estimates teach more than a simulation plot alone?

**Answer.** The exact curve gives a benchmark independent of random sampling. It lets us see the scale of Monte Carlo variation and detect systematic implementation errors. A simulation plot alone does not tell us whether its pattern is the correct prediction of the model.

# Chapter 10

# Run, inspect, and reproduce the project

## 10.1 Three things that are easy to confuse

The **source code** is the set of instructions in files ending in `.jl`. The **results** are the numbers those instructions write after a run. The **guide** is the document you are reading. Editing a sentence in the guide does not run the model. Running the model does not automatically turn every new result into a new PDF.

We therefore use separate steps: run the Julia code, prepare the document's inputs, and compile the LaTeX document. This chapter explains the steps and then walks through the scripts that connect them.

## 10.2 Open the project folder in a terminal

A **terminal** accepts text commands for the operating system. It is different from the Julia prompt. The commands below belong in the terminal. If you see the Julia prompt `julia>`, leave that session first or open a separate terminal.

Extract the supplied project archive if you are starting from the archive. Locate the folder named `moran_project`. Inside it, you should find folders named `src`, `scripts`, `test`, `results`, and `guide`.

The command `cd` changes the terminal's current directory. The next line uses a placeholder path. Replace `/path/to/moran_project` with the actual path to your extracted folder. Keep the quotation marks if the path contains spaces.

```
cd "/path/to/moran_project"
```

To check where the terminal is looking, use these two commands:

```
pwd
ls
```

The first prints the current directory. The second lists its contents. If you do not see `scripts`

and `src`, you are probably looking at the wrong folder. This is a file-location problem, not a population-genetics result.

## 10.3 Run the small demonstration first

Run this command from the project folder:

```
julia --project=. scripts/demo.jl
```

The word `julia` starts Julia. The option `-project=.` selects the current directory as the Julia project environment. A single dot means "the current directory". The final part names the script to execute. The model uses Julia standard libraries; it does not require installing a separate plotting package to generate its data.

The demonstration first prints the probabilities for the five-individual example. They should be 0.3 for an increase, 0.2 for a decrease, and 0.5 for no change. It then shows a short specified sequence of draws, followed by a seeded repeated experiment.

The specified draws are a teaching example. They are not a random sample used to estimate fixation probability. The repeated experiment that follows uses the random-number generator.

## 10.4 Read every part of the demonstration script

The following blocks are inserted directly from `scripts/demo.jl`. They are not separately maintained copies of the code. The numbers at the left are the line numbers in the original file.

### 10.4.1 Find the model file and load the libraries

```
1 # Run from the project folder: julia --project=. scripts/demo.jl
2 include(joinpath(@__DIR__, "..", "src", "MoranModel.jl"))
3 using .MoranModel
4 using Random
5 using Printf
```

Line 1 is a comment. The initial hash sign tells Julia that the rest of the line is an explanation for people.

Line 2 loads the model file. The macro `@__DIR__` means the directory containing this script. The function `joinpath` joins directory and file names using the operating system's path convention. The component `..` means "go to the parent directory". Starting from the `scripts` folder, the constructed path goes up to the project folder, then into `src`, then to `MoranModel.jl`. The function `include` evaluates that file so its module becomes available.

Line 3 makes the exported names from the local `MoranModel` module available in this script. The initial dot in `.MoranModel` identifies a module already loaded in the current context. Line 4 loads the random-number tools. Line 5 loads tools for formatted printing. The Julia documentation describes these language and standard-library features [2].

### 10.4.2 Set the toy state and follow specified draws

```
7  function main()
8      N = 5
9      i = 2
10     s = 0.5
11     probabilities = transition_probabilities(N, i, s)
12     println("ONE EVENT: N=5, i=2, s=0.5")
13     @printf("up = %.3f, down = %.3f, stay = %.3f\n", probabilities.up, probabilities.down,
       probabilities.stay)
14     println("\nA HAND TRACE (specified draws, not a random sample)")
15     draws = [0.12, 0.72, 0.41, 0.95, 0.10, 0.03]
16     for (event, u) in enumerate(draws)
17         probabilities = transition_probabilities(N, i, s)
18         i = next_count(i, probabilities.up, probabilities.down, u)
19         @printf("event %d: draw %.2f -> i = %d\n", event, u, i)
20     end
```

Line 7 begins a function called `main`. This keeps the script's work inside one function. Lines 8–10 assign the population size, current A count, and selection advantage. Line 11 passes those three values to the model's transition-probability function.

The returned value contains named fields. Thus `probabilities.up` selects its increase probability, `probabilities.down` its decrease probability, and `probabilities.stay` its no-change probability. Lines 12–13 print those values for a reader to inspect.

The macro `@printf` prints values according to a format string. In that string, `%.3f` means a decimal number with three digits after the decimal point. The character sequence `\n` means a new line. Formatting affects the displayed number, not the underlying calculation.

Line 15 creates an array: an ordered collection of six chosen draws. Square brackets enclose its entries. Line 16 uses `enumerate` to obtain two values for each entry: its position and the draw itself. The variable `event` receives the position, and `u` receives the draw.

Line 17 recomputes the probabilities using the *current* count. Line 18 chooses the next count and assigns it back to `i`. This assignment is what allows the count to evolve. Line 19 prints the event number, draw, and new count. In its format, `%d` means an integer, while `%.2f` means two digits after the decimal point. Line 20 ends the loop.

> At the first hand-trace event, the count is two and the increase threshold is 0.3. The first specified draw is 0.12, which is smaller than 0.3, so the count increases to three. Before the next draw, line 17 calculates new probabilities using three A individuals. The program does not keep using the original thresholds from the two-individual state.

### 10.4.3 Repeat complete runs and summarise them

```
22     println("\nA SEEDED EXPERIMENT: N=100, i0=1, s=0.01")
23     rng = MersenneTwister(20261027)
24     repetitions = 2000
25     fixations = 0
26     for repetition in 1:repetitions
27         outcome = simulate_fixation(rng, 100, 1, 0.01)
28         fixations += outcome.fixed
29     end
```

```
    interval = wilson_interval(fixations, repetitions)
    @printf("fixations: %d / %d\n", fixations, repetitions)
    @printf("estimate: %.6f\n", fixations / repetitions)
    @printf("95%% Wilson interval: [%.6f, %.6f]\n", interval.lower, interval.upper)
    @printf("exact probability: %.6f\n", fixation_probability(100, 1, 0.01))
end

main()
```

Line 23 creates a random-number generator with a recorded seed. `MersenneTwister` is the generator's name; the integer in parentheses initialises it. The variable `rng` stores the generator. Line 24 sets the number of independent simulation replicates to two thousand. Line 25 starts the fixation counter at zero.

Line 26 repeats the following block once per replicate. Line 27 starts each run at population size one hundred, one A individual, and advantage $s = 0.01$. Each run continues until loss or fixation. The next replicate starts again from one A individual. It does not inherit the preceding replicate's final population.

The result has a Boolean field called `fixed`. A **Boolean** is either `true` or `false`. In this addition, Julia treats `true` as one and `false` as zero. Therefore `fixations += outcome.fixed` adds one for a fixation and adds zero for a loss. The operation `+=` means "add to the existing value and store the result back".

After the loop ends, line 30 calculates the Wilson interval. Lines 31–34 print the fixation count, its estimated probability, its interval, and the exact mathematical probability. The format `%.6f` uses six digits after the decimal point. The sequence `%%` prints a literal percent sign within an `@printf` format string.

Line 35 ends the definition of `main`. Line 37 calls it. Defining a function describes its instructions; calling the function carries them out.

## 10.5 Run the automated checks

After the demonstration, run:

```
julia --project=. test/runtests.jl
```

The checks ask whether the code obeys known properties of the model. They cover valid inputs, boundary states, transition probabilities, exact fixation probabilities, the independent backward-equation calculation, and other specified behaviour. They do not prove that every real population follows the Moran model. They assess whether this implementation does the things we say it does.

Read a failure message before changing anything. A failing expected-value check and a command that cannot locate a file are different problems. The former may indicate incorrect code or an incorrect test. The latter means the requested file was not found.

## 10.6 Choose a quick study or the full study

For a faster practice run, use:

```
julia --project=. scripts/run_study.jl --quick
```

This runs one thousand replicates per parameter combination and saves data under `results/demo`. It exercises the complete data-generation procedure with a smaller Monte Carlo sample.

For the study used in this guide, omit the flag:

```
julia --project=. scripts/run_study.jl
```

The default is twenty thousand replicates per combination. The full data go under `results/full`. There is no `-full` flag; full mode is simply the default. The script recognises `-quick` and rejects unrecognised arguments rather than silently guessing what they mean.

The grid contains four population sizes and seven selection advantages, giving $4 \times 7 = 28$ combinations. We call one combination a **cell**. The full run therefore contains $28 \times 20000 = 560000$ complete fixation experiments. The quick run contains $28 \times 1000 = 28000$. The deterministic calculations and small set of display trajectories are generated in both modes.

Running a mode again replaces that mode's generated data files. If you want to retain an experimental variant for comparison, first give the copied project or its copied result folder a descriptive name.

## 10.7 Read the study script: loading and writing a CSV row

```
1  # Run from the project folder: julia --project=. scripts/run_study.jl
2  # For a faster check: julia --project=. scripts/run_study.jl --quick
3  include(joinpath(@__DIR__, "..", "src", "MoranModel.jl"))
4  using .MoranModel
5  using Random
6  using Printf
7  using Dates
8
9  # 1. A small helper writes one row of comma-separated values.
10 function write_row(file, values)
11     println(file, join(values, ","))
12 end
```

The loading instructions match the demonstration. This script also loads `Dates`, which provides the finishing timestamp.

The helper function on lines 10–12 writes one row of values. Its argument `file` identifies the open output file. Its argument `values` contains the entries to write. The function `join(values, ",")` turns those entries into text separated by commas. The function `println(file, ...)` writes that text to the file and ends the line.

The resulting format is **CSV**, meaning comma-separated values. Each line is a row; each comma separates columns. A spreadsheet program can read these files, and they can also be opened as

plain text. Our rows contain numbers or simple labels that do not themselves contain commas. The helper is sufficient for these controlled outputs; it is not a general CSV parser for arbitrary quoted text.

## 10.8 Read the study script: the fixation grid

```
15 function run_fixation_grid(output_directory, repetitions, base_seed)
16     output_path = joinpath(output_directory, "fixation.csv")
17     open(output_path, "w") do file
18         println(file, "N,s,i0,repetitions,fixations,estimate,lower,upper,exact,standard_error
       ,events_mean,seed")
19         cell = 0
20         for N in [20, 50, 100, 200]
21             for s in [0.0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1]
22                 cell += 1
23                 seed = base_seed + cell
24                 rng = MersenneTwister(seed)
25                 fixations = 0
26                 total_events = 0
27                 for repetition in 1:repetitions
28                     outcome = simulate_fixation(rng, N, 1, s)
29                     fixations += outcome.fixed
30                     total_events += outcome.events
31                 end
32                 estimate = fixations / repetitions
33                 interval = wilson_interval(fixations, repetitions)
34                 exact = fixation_probability(N, 1, s)
35                 standard_error = sqrt(estimate * (1 - estimate) / repetitions)
36                 write_row(file, (N, s, 1, repetitions, fixations, estimate,
37                     interval.lower, interval.upper, exact, standard_error,
38                     total_events / repetitions, seed))
39                 flush(file)
40                 @printf("cell %2d/28: N=%3d s=%.3f estimate=%.6f exact=%.6f\n", cell, N, s,
       estimate, exact)
41             end
42         end
43     end
44 end
```

### 10.8.1 Open the file and label its columns

Line 15 defines a function taking three inputs: where to save results, how many replicates to run, and a base seed. Line 16 constructs the name `fixation.csv`. Line 17 opens that file for writing. The string `"w"` specifies write mode.

The syntax `do file` starts a **do block**. It gives the opened file the local name `file` within the block. Julia closes the file when the block finishes, including when an error interrupts the block. Line 18 writes the header: the names of the data columns.

### 10.8.2 Visit all parameter combinations

Line 19 starts the cell counter at zero. Line 20 selects a population size from an array. Inside that loop, line 21 selects each advantage from a second array. These are **nested loops**: all seven advantages are used for one population size before moving to the next size.

Line 22 increments the cell number. Line 23 makes its seed by adding the cell number to the base seed. Line 24 constructs the generator for this cell. This gives every cell a recorded seed without resetting the generator at the start of every replicate. The generator continues its sequence as replicates run.

### 10.8.3 Run each replicate to an absorbing state

Lines 25–26 initialise two counters. One counts fixations; the other sums the number of event attempts used by the complete runs. Lines 27–31 repeat the experiment. The initial A count in line 28 is always one. Line 29 adds the Boolean fixation result. Line 30 adds that run's event count, including unchanged attempts.

There is no display horizon in this loop. It continues each replicate until one of the two absorbing boundaries is reached. Thus the fixation estimate is not contaminated by treating an unfinished display trajectory as a loss.

### 10.8.4 Summarise and save the cell

Line 32 divides fixations by replicates. Line 33 calculates the interval. Line 34 evaluates the exact formula, independently of the simulation count. Line 35 computes the usual plug-in standard-error estimate for a proportion: the square root of the estimated probability times one minus that estimate, divided by the number of replicates. This standard error describes the approximate Monte Carlo sampling scale. It is not another biological parameter. The plotted confidence interval uses the Wilson calculation instead.

Lines 36–38 pass one **tuple** of results to the row writer. A tuple is an ordered group of values, written here in parentheses. The order must match the header. In order, the columns are:

- `N`, `s`, and `i0`: population size, advantage, and initial A count;
- `repetitions` and `fixations`: replicate count and observed fixation count;
- `estimate`, `lower`, and `upper`: estimated probability and Wilson interval endpoints;
- `exact` and `standard_error`: the mathematical target and estimated Monte Carlo standard error;
- `events_mean` and `seed`: average number of event attempts to either absorbing outcome, and the cell's seed.

The average event count includes both fixation and loss runs. It is not the average time conditional on successful fixation.

Line 39 uses `flush` to send buffered output to the file during the study. Line 40 prints a progress update. In its format, `%2d` reserves at least two character positions for an integer and `%3d` reserves at least three. The widths improve alignment; they do not limit the numbers. The remaining `end` statements finish the inner loop, outer loop, file block, and function.

## 10.9 Read the study script: compare initial conditions

```
47 function write_initial_frequency_comparison(output_directory)
48     open(joinpath(output_directory, "initial_frequency.csv"), "w") do file
49         println(file, "N,s,i0,initial_frequency,exact_single,exact_ten_percent,
       relative_to_neutral_single")
50         for N in [20, 50, 100, 200]
51             for s in [0.0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1]
52                 i0 = div(N, 10)
53                 single = fixation_probability(N, 1, s)
54                 multiple = fixation_probability(N, i0, s)
55                 write_row(file, (N, s, i0, i0 / N, single, multiple, N * single))
56             end
57         end
58     end
59 end
```

This block opens `initial_frequency.csv` and loops over the same population sizes and advantages. It calculates probabilities with the exact formula, so there is no random sampling in this block.

Line 52 uses Julia's `div(N, 10)` function for integer division, to calculate `N` divided by ten as an integer count. All four supplied population sizes are divisible by ten. Thus the result is exactly ten percent of the population: two, five, ten, or twenty A individuals.

Line 53 calculates fixation probability from one copy. Line 54 calculates it from the ten-percent count. The row in line 55 records both answers, the ten-percent initial count and frequency, and the single-copy fixation probability relative to its neutral baseline.

For the last comparison, the neutral single-copy probability is $1/N$. Dividing the selected probability by $1/N$ is the same as multiplying it by $N$, which explains `N * single`. A value of two in that column means twice the neutral odds. It does not mean a probability of two.

## 10.10 Read the study script: check the approximation

```
62 function write_approximation_comparison(output_directory)
63     open(joinpath(output_directory, "approximation.csv"), "w") do file
64         println(file, "N,s,Ns,exact,approximation,relative_error")
65         selection_values = 10.0 .^ range(log10(1e-5), log10(0.5), length=121)
66         for N in [20, 50, 100, 200]
67             for s in selection_values
68                 exact = fixation_probability(N, 1, s)
69                 approximation = s / (-expm1(-N * s))
70                 relative_error = abs(approximation - exact) / exact
71                 write_row(file, (N, s, N * s, exact, approximation, relative_error))
72             end
73         end
74     end
75 end
```

This block writes `approximation.csv`. Line 65 creates 121 positive selection values from $10^{-5}$, which is 0.00001, up to 0.5. They are evenly spaced on a logarithmic scale, not evenly spaced by ordinary addition. That gives detail near very small advantages as well as larger ones.

The function `log10` is the logarithm to base ten: it tells us the power to which ten must be raised. The function `range` produces evenly spaced exponents between the two endpoint logarithms. Its `length=121` argument specifies the number of values. The operator `.^` raises ten to *each* exponent in that range. The dot means to apply the operation element by element; this is called **broadcasting**.

Lines 66–67 loop over sizes and these finer selection values. Line 68 calculates the exact single-copy answer. Line 69 calculates the weak-selection approximation explained earlier. The use of `-expm1(-N * s)` calculates $1 - \exp(-Ns)$ accurately near zero. Line 70 subtracts the exact answer from the approximation and divides by the exact answer. This is a *signed* relative error: positive means the approximation is above the exact answer, and negative means it is below.

The values in this block are mathematical comparisons, not estimated simulation frequencies. Increasing the number of replicates cannot repair an inaccurate approximation. The loop starts above zero; the neutral limit is explained separately instead of evaluating a zero-over-zero expression.

## 10.11 Read the study script: save example trajectories

```
function write_trajectories(output_directory, base_seed)
    N = 100
    max_events = 30000
    sample_every = 100
    paths_per_condition = 8
    condition = 0
    summary_path = joinpath(output_directory, "trajectory_summary.csv")
    open(summary_path, "w") do summary
        println(summary, "N,i0,s,path,seed,absorbed,absorbed_at,final_count")
        for i0 in [1, 10]
            for s in [0.0, 0.01, 0.1]
                condition += 1
                paths = Vector{Vector{Int}}()
                for path_number in 1:paths_per_condition
                    seed = base_seed + 100000 + 100condition + path_number
                    trajectory = simulate_trajectory(MersenneTwister(seed), N, i0, s,
    max_events)
                    push!(paths, trajectory.counts)
                    absorbed_at = trajectory.absorbed_at
                    if absorbed_at === nothing
                        absorbed_at = -1
                    end
                    write_row(summary, (N, i0, s, path_number, seed,
                        trajectory.absorbed, absorbed_at, trajectory.counts[end]))
                end
                deterministic = deterministic_trajectory(N, i0, s, max_events)
                selection_label = replace(string(s), "." => "p")
                filename = "trajectory_N$(N)_i$(i0)_s$(selection_label).csv"
                open(joinpath(output_directory, filename), "w") do file
                    println(file, "event,generation,path_1,path_2,path_3,path_4,path_5,path_6
    ,path_7,path_8,deterministic")
                    for event in 0:sample_every:max_events
                        row = Float64[event, event / N]
                        for counts in paths
                            push!(row, counts[event + 1] / N)
                        end
```

```
                        push!(row, deterministic[event + 1])
                        write_row(file, row)
                    end
                end
            end
        end
    end
end
```

### 10.11.1 Set the display experiment

Lines 79–82 choose a population of one hundred, a display horizon of thirty thousand event attempts, output every one hundred attempts, and eight paths per condition. A **path**, or trajectory, is the sequence of counts in one run. Line 83 starts a condition counter. Lines 84–86 open the trajectory summary file and label its columns.

Lines 87–88 combine initial counts of one and ten with three advantages: zero, 0.01, and 0.1. There are six conditions in total. Line 89 updates the condition number.

### 10.11.2 Store eight paths without selecting their outcomes

Line 90 creates an initially empty vector whose entries will themselves be vectors of integer counts. The notation `Vector{Vector{Int}}()` means "a vector of integer vectors". Line 91 repeats the path-generation block eight times.

Line 92 assigns a seed using the base seed, the condition number, and the path number. The expression `100condition` means `100 * condition`; Julia allows a numerical coefficient directly before a variable name. The large fixed offset keeps these path seeds separate from the fixation-grid seeds. Line 93 creates a generator with that seed and simulates the path. Line 94 appends its count vector to the collection. The exclamation mark in `push!` indicates that it changes the collection it receives.

These are predetermined seeds and path counts. The script does not search for an appealing successful trajectory and discard the others.

Lines 95–98 inspect when the trajectory first reached a boundary. The value `nothing` means no absorption event was recorded before the display horizon. The comparison `=== nothing` tests for that particular missing-value marker. For the CSV file, the code replaces it with $-1$. This negative number is a label for "not yet absorbed within the display", not a negative event time and not a loss.

Lines 99–100 write the condition, path number, seed, absorption indicator, absorption time or marker, and final count. The expression `trajectory.counts[end]` retrieves the last entry of the counts vector. The `end` inside square brackets means "the final index" here; it is not the instruction that closes a loop.

### 10.11.3 Save frequencies on a readable time grid

Line 102 calculates the deterministic comparison for the same starting state and horizon. Line 103 turns the selection value into text and replaces its decimal point with the letter `p`. In `"." => "p"`, the arrow specifies a replacement rule. Thus `0.01` becomes the filename label `0p01`.

Line 104 inserts the parameter values into a filename. Inside a Julia string, `$(N)` means "evaluate this expression and insert its text here". This is **string interpolation**. For example, one generated name is `trajectory_N100_i1_s0p01.csv`.

Lines 105–106 open the path file and write its header. Line 107 uses this range:

```
0:sample_every:max_events
```

Its three components mean starting value, step size, and final allowed value. Here it visits events zero, one hundred, two hundred, and so on, through thirty thousand.

Line 108 creates the beginning of a numerical row. The type `Float64` requests ordinary double-precision numerical entries. The first entry is the event number; the second is the event number divided by the population size. That second number is our generation-like time unit. At one hundred events and $N = 100$, it equals one. No synchronous replacement of all individuals is implied.

Lines 109–111 append each path's frequency at that event. The array index is `event + 1` because array position one contains the initial state at event zero. Dividing the stored count by `N` converts it into a frequency. Line 112 appends the deterministic frequency, which is already a frequency and must not be divided by `N` again. Line 113 writes the row. The remaining `end` statements close the loops, file blocks, and function.

The stored histories extend to the display horizon. After absorption, a history repeats its absorbing count. That is why a path can stay exactly at zero or one for the rest of the displayed time.

## 10.12 Read the study script: record the run information

```
122 function write_metadata(output_directory, repetitions, base_seed, elapsed_seconds, quick)
123     open(joinpath(output_directory, "metadata.txt"), "w") do file
124         println(file, "Moran model simulation study")
125         println(file, "Julia version: ", VERSION)
126         println(file, "Random generator: Random.MersenneTwister")
127         println(file, "Base seed: ", base_seed)
128         println(file, "Cell seeds: base_seed + cell, in N-then-s loop order")
129         println(file, "Trajectory seeds: base_seed + 100000 + 100*condition + path_number")
130         println(file, "Population sizes: 20, 50, 100, 200")
131         println(file, "Selection advantages: 0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1")
132         println(file, "Initial copies for simulation study: 1")
133         println(file, "Repetitions per cell: ", repetitions)
134         println(file, "Quick demonstration mode: ", quick)
135         println(file, "No external Julia packages are required.")
136         println(file, "Every fixation experiment runs until loss or fixation.")
137         println(file, "Trajectory display horizon: 30000 events; 100 events per plotted
    generation.")
138         println(file, "trajectory_summary absorbed_at=-1 means unfinished at the display
    horizon.")
139         println(file, "Elapsed seconds: ", elapsed_seconds)
140         println(file, "Completed UTC: ", Dates.now(Dates.UTC))
141     end
142 end
```

This function writes `metadata.txt`, a plain-text description of the computation. The file records

the Julia version, random-number generator, base seed, seed conventions, parameter grid, replicate count, and mode. The constant `VERSION` contains the running Julia version.

The file also records the stopping rules. This matters because the fixation experiment runs until absorption, whereas displayed trajectories have a finite horizon. A future reader should not have to guess which rule was used.

Line 139 writes the elapsed time supplied by the main function. Line 140 writes the completion timestamp in UTC using `Dates.now(Dates.UTC)`. A timestamp identifies when the computation finished; elapsed seconds measure how long it took. They serve different purposes. Runtime depends on the computer and Julia compilation state, so it is not a biological result.

## 10.13 Read the study script: the main function

```
145 function main()
146     quick = "--quick" in ARGS
147     unknown = setdiff(ARGS, ["--quick"])
148     isempty(unknown) || error("Unknown arguments: $(join(unknown, ", "))")
149     repetitions = 20000
150     mode = "full"
151     if quick
152         repetitions = 1000
153         mode = "demo"
154     end
155     base_seed = 20261027
156     output_directory = normpath(joinpath(@__DIR__, "..", "results", mode))
157     mkpath(output_directory)
158     start_time = time()
159     run_fixation_grid(output_directory, repetitions, base_seed)
160     write_initial_frequency_comparison(output_directory)
161     write_approximation_comparison(output_directory)
162     write_trajectories(output_directory, base_seed)
163     elapsed_seconds = time() - start_time
164     write_metadata(output_directory, repetitions, base_seed, elapsed_seconds, quick)
165     @printf("Finished in %.2f seconds. Data: %s\n", elapsed_seconds, output_directory)
166 end
167
168 main()
```

Line 146 checks whether the string `"-quick"` is present in `ARGS`, Julia's list of command-line arguments supplied after the script name. Line 147 uses `setdiff` to identify supplied arguments outside the permitted list. Line 148 checks whether that unrecognised list is empty.

The operator `||` means short-circuit "or". If the left expression is true, Julia does not evaluate the right expression. Thus an empty unrecognised-argument list allows execution to continue; otherwise `error` reports which arguments were not understood. This is an input check, not a statistical test.

Lines 149–150 set the default replicate count and output mode. Lines 151–154 replace them with one thousand and `"demo"` when quick mode was requested. Line 155 sets the base seed. Line 156 constructs the output directory relative to the script's location; `normpath` simplifies path components such as `..`. Line 157 uses `mkpath` to create the directory if needed.

Line 158 records the clock time before the work begins. Lines 159–162 call the four data-

generation functions in order. Line 163 subtracts the starting clock time from the current clock time, producing elapsed seconds. Line 164 writes the metadata. Line 165 prints a completion message; its `%s` placeholder prints text. Line 166 ends the function definition, and line 168 calls the function to run the script.

## 10.14 Turn full results into the document inputs

After generating the full results, run:

```
julia --project=. scripts/prepare_guide.jl
```

This preparation step reads the full-study CSV files and writes the numerical and figure inputs used by the LaTeX document. It does not run a new set of fixation simulations. This separation ensures that the written results and figures refer to the saved study data.

The guide uses the full results, not the smaller quick demonstration. If you rerun only quick mode, you have practised the pipeline but have not replaced the full tables used in the guide. To update the guide with a changed full experiment, rerun full mode and this preparation step.

## 10.15 Compile or open the LaTeX guide

If Tectonic is available, run these commands from the project folder:

```
cd guide
tectonic --keep-logs main.tex
```

The first command moves into the guide folder. The argument `main.tex` identifies the main LaTeX source file. Tectonic writes `main.pdf` in that folder. The option keeps a compilation log that can help diagnose errors.

You can also open the `guide` project folder in the Codex LaTeX editor and use its PDF preview. The LaTeX compilation step uses the saved source and prepared inputs. It does not execute Julia automatically.

An online editor such as Overleaf can be used for editing LaTeX too. The preparation helper makes the guide folder self-contained: it copies the actual runnable source into `guide/code` and the plotting data into `guide/generated`. Upload the contents of the whole guide folder, choose `main.tex` as the main document, and use pdfLaTeX. Uploading only the main file would omit the chapters and supporting inputs. Edit the original Julia files and rerun the helper to refresh their copies before recompiling.

## 10.16 Make a small change and understand what it changes

Keep an unchanged copy of the supplied project as a reference. In a working copy, change one setting at a time. First make a prediction in words. Then run the demonstration or quick study. Finally compare the observation with your prediction and the exact result.

For example, change only the demonstration's selection advantage from 0.5 to zero in the toy-

state block. Keep $N = 5$ and $i = 2$. Predict that the increase and decrease probabilities will become equal. Their new values should be 0.24, while the no-change probability should be 0.52. This is a stronger learning exercise than changing several parameters and guessing which one caused the difference.

Changing a printed label does not change a function's input. Check the actual arguments passed to the function. In particular, the seeded experiment in the demonstration passes its own numbers explicitly, so changing the earlier toy variables does not change that later experiment.

If you add a new population size to the study, remember that the exact ten-percent comparison currently uses integer division by ten. Choose a size divisible by ten if you want to preserve that initial frequency exactly. Changes to the scientific question may require changes to more than one output label or loop; work through them deliberately.

## 10.17 Diagnose a message before drawing a conclusion

- **The terminal says Julia was not found.** The operating system could not locate the Julia executable. Check the Julia installation and terminal path. No simulation result has been produced.
- **A script or source file cannot be opened.** Check the current folder and the extracted directory layout. Keep the project folders together.
- **An `ArgumentError` describes an input.** Read that description. The model requires an integer population size of at least two, an integer count between zero and the size, and a finite nonnegative advantage. A rejected input is not evidence about allele behaviour.
- **An unknown command-line argument is reported.** Use no flag for full mode or `-quick` for the demonstration mode.
- **A simulation estimate differs from the exact probability.** Compare the difference with its sampling uncertainty. A small discrepancy is expected. Repeated systematic discrepancies much larger than that scale deserve investigation.
- **A display path has absorption time $-1$.** It had not reached a boundary by the displayed horizon. It was not automatically counted as a loss in the separate fixation experiment.

**Question 1.** Which command produces the twenty-thousand-replicate study?
**Answer.** Run `julia -project=. scripts/run_study.jl` from the project folder, without `-quick`.
**Question 2.** You edited the full-study settings, ran only quick mode, and recompiled the guide. Should its full-study tables have changed?
**Answer.** No. Quick mode writes the demonstration results. Rerun the full study, run `scripts/prepare_guide.jl`, and then compile the document to update the full-study presentation.
**Question 3.** A trajectory file reports event 500 at generation 5 for $N = 100$. What calculation produced that time?
**Answer.** The script divided the event count by the population size: $500/100 = 5$. It is a model time unit, not five synchronised replacements of every individual.

# Appendix A

# A reference sheet and a teach-back check

## A.1 The main biological words

**Allele**
A version of the genetic locus being followed. Our model has two versions, A and B.

**Haploid model**
Each individual has one copy of the locus, so it has either A or B in this model.

**Focal type**
The type whose fate we follow. We call it A.

**Resident type**
The alternative type already present. We call it B.

**Selection**
A difference in expected reproductive success between the types. Here it enters through parent-selection weights.

**Genetic drift**
Random change in allele frequency due to finite random sampling. It occurs even when the weights are equal.

**Neutrality**
Equal reproductive weights. In our parameter convention, the advantage is zero.

**Loss**
No A individuals remain. The population itself still exists.

**Fixation**
Every individual in the modelled population is A.

**Absorbing state**
A state that cannot be left under the rules. With no mutation, zero A individuals and all A individuals are absorbing.

**Well mixed**
The rules do not assign spatial restrictions to parent or replacement selection.

**Trajectory or path**

The ordered sequence of states from one run.

**Replicate**

A new complete run from the specified initial condition, using further random draws.

## A.2 Read the mathematical notation in ordinary words

In the table below, each symbol is followed by its meaning. The same letters must retain the same definitions when you explain the project.

| Symbol | Ordinary meaning |
|---|---|
| $N$ | The total number of individuals. It stays constant during a run. |
| $i$ | The current number of A individuals. It changes during a run. |
| $i_0$ | The number of A individuals at the start. In the main fixation experiment, this is one. |
| $N - i$ | The current number of B individuals. |
| $x = i/N$ | The current fraction, or frequency, of A. |
| $s$ | The relative reproductive advantage of A in this guide's convention. The main code accepts finite, nonnegative values. |
| $r = 1 + s$ | The reproductive weight of one A individual when one B individual has weight one. |
| $W_i = ri + N - i$ | The total reproductive weight of the population at count $i$. |
| $T_i^+$ | The chance that the next event increases the A count by one. |
| $T_i^-$ | The chance that the next event decreases the A count by one. |
| $T_i^0$ | The chance that the next event leaves the A count unchanged. |
| $u_i$ | The probability of eventual A fixation, starting from count $i$. This concerns the whole future run. |
| $d_i = u_i - u_{i-1}$ | The change in fixation probability between two neighbouring starting counts. |
| $\sigma = \log(1 + s)$ | The log-fitness difference. It is useful for the stable calculation and reference-paper comparison. |
| $R$ | The number of repeated complete simulation runs. |
| $K$ | The number of those runs in which A fixes. |
| $\widehat{p} = K/R$ | The estimated fixation probability from the simulated runs. The hat marks it as an estimate. |
| $\mathbb{E}[Y]$ | The probability-weighted average of the random quantity called $Y$. |
| $\Delta x$ | The change in allele frequency during one event. |
| $\Pr(B \mid A)$ | The probability of a probability event called $B$, given that an event called $A$ occurred. These letters are event labels in this notation. |
| $\sum$ | Add the terms specified by the expression's index and limits. |
| $\approx$ | Approximately equal; an approximation or rounded value is being used. |

The symbols used inside a mathematical derivation and the variable names used inside a function need not be identical. For example, the model's function `wilson_interval(k, n)` uses `k` and `n` for a success count and replicate count. In that function, `n` is not the population size $N$. The argument's meaning comes from the function's definition and the value passed to it.

## A.3 Programming terms in plain language

**Variable**
A name referring to a value. Assignment gives that name its current value.

**Function**
A named set of instructions that takes inputs and can return an output. Defining a function does not by itself run its body.

**Argument**
A value supplied when calling a function. Its position or keyword tells the function how to use it.

**Return value**
The result a function gives back to its caller.

**Loop**
An instruction to repeat a block. A `for` loop visits a specified sequence; a `while` loop continues while a condition remains true.

**Conditional**
An `if`, `elseif`, or `else` choice that selects instructions according to a condition.

**Boolean**
A truth value, either `true` or `false`.

**Array or vector**
An ordered collection of entries. Julia's first ordinary array index is one, not zero.

**Index**
A position used to select an entry from a collection.

**Tuple**
An ordered group of values. A named tuple also gives fields descriptive names, accessed with a dot.

**Module**
A container grouping related Julia definitions and controlling which names are exported for convenient use.

**Standard library**
A library supplied with Julia, such as `Random`, `Printf`, or `LinearAlgebra`.

**Random-number generator**
An algorithm that supplies the sequence of numbers used to sample random outcomes in the simulation.

**Seed**
A recorded initial setting for the random-number generator. Reproducibility also depends on the code and software versions.

**Broadcasting**
Applying an operation separately to the entries of a collection. A dot on an operator, such as `.^`, requests this.

**String**
: Text held as a value, commonly written in double quotation marks. A filename or message can be a string.

**CSV**
: A text-file format with comma-separated columns and one row per line in our controlled outputs.

**Working directory**
: The folder the terminal treats as the current location for relative commands and paths.

**Validation**
: Checking that supplied values meet a function's stated requirements before using them.

**Test**
: A specified check that compares behaviour with an expected property or independently known answer.

## A.4 Connect mathematical ideas to the code

| Mathematical or modelling task | Julia function or script |
|---|---|
| Check whether the inputs describe an allowed population | `validate_parameters` |
| Calculate the three next-event probabilities | `transition_probabilities` |
| Turn a uniform draw into increase, decrease, or no change | `next_count` |
| Store one history to a chosen display horizon | `simulate_trajectory` |
| Run until loss or fixation and return its outcome | `simulate_fixation` |
| Evaluate the exact fixation probability | `fixation_probability` |
| Solve the backward equations independently | `backward_probabilities` |
| Calculate an interval for a sampled success proportion | `wilson_interval` |
| Generate the deterministic mean-field comparison | `deterministic_trajectory` |
| Print the toy example and a small repeated experiment | `scripts/demo.jl` |
| Generate the grid, analytical comparisons, paths, and metadata | `scripts/run_study.jl` |
| Convert full-study outputs into document inputs | `scripts/prepare_guide.jl` |
| Check specified implementation properties | `test/runtests.jl` |

## A.5 Five distinctions to keep clear

1. **An event probability and a fixation probability.** The increase probability describes the next attempt. Fixation probability describes the eventual absorbing outcome.
2. **A parameter and a state.** Population size and advantage are fixed settings. The current A count is the changing state.
3. **A deterministic prediction and an average of simulations.** The deterministic curve follows a mean-field recurrence. It is not generally the exact average of the random trajectories.
4. **An estimate and an exact answer.** The simulated proportion varies across finite samples. The exact probability is calculated from the model's mathematical rules.
5. **A display horizon and an absorption stopping rule.** A displayed path can still be unfinished at its final plotted event. The complete fixation experiment continues until its outcome is known.

## A.6 Teach the model back without looking

Try answering the following questions aloud before reading the answer. You have understood the project when you can connect each answer to a biological rule, an equation, and a part of the code.

**1. What does one model event do?**
**Answer.** It chooses a parent proportional to reproductive weight, independently chooses a current individual uniformly for replacement, and places a copy of the parent in that individual's place. The total number of individuals stays fixed.
**2. Why multiply two fractions when calculating an increase?**
**Answer.** An increase needs both an A parent and a B replacement. Given the current state, those selections are independent. Their joint probability is the parent probability times the replacement probability.
**3. In the five-individual example, why is the chance of an A parent one half even though only two individuals are A?**
**Answer.** With A weight 1.5, the total A weight is $2 \times 1.5 = 3$. The three B individuals have total weight three too. A supplies three of the six weight units, so its parent probability is $3/6 = 1/2$.
**4. Why can the count stay unchanged after reproduction?**
**Answer.** An A offspring can replace A, or a B offspring can replace B. These are real event attempts even though the type count is unchanged.
**5. Why can an advantageous allele be lost?**
**Answer.** A reproductive advantage makes an A parent more likely relative to a B individual of the same abundance. It does not force A to reproduce before being replaced. A sequence of chance events can reach zero A individuals, especially when A begins rare.

**6. How do you estimate fixation probability from the code?**

**Answer.** Start many complete runs at the same initial condition, count how many end with all A, and divide that count by the number of runs. Every replicate in this estimate runs until its absorbing outcome is known.

**7. What does a confidence interval add?**

**Answer.** It reports uncertainty from observing a finite number of random outcomes. A nominal 95 percent interval procedure is designed for approximately 95 percent coverage over repeated batches under its assumptions. It does not assign a 95 percent probability to the fixed true parameter after this particular interval has been calculated.

**8. Why is the neutral answer the initial frequency?**

**Answer.** In the backward-equation derivation, equal reproductive weights make all neighbouring fixation-probability differences equal. Starting from zero at count zero and ending at one at count $N$, the increment must be $1/N$ per extra copy. At count $i$, the answer is therefore $i/N$.

**9. What information lets us condition on just the next step?**

**Answer.** The current count and fixed parameters determine all future transition rules. The previous path is not needed. This is the Markov property. After the next event, we use the fixation probability associated with the resulting count.

**10. Why is solving the backward equations a useful check?**

**Answer.** The solver obtains the probabilities from a set of linear equations rather than evaluating the closed formula. Agreement tests whether two different calculations of the same model match.

**11. What does the deterministic curve leave out?**

**Answer.** It replaces random event outcomes with a smooth mean-field change. It does not represent chance early loss from a positive initial frequency. Its state can be fractional, whereas an actual A count is an integer.

**12. Does twice the neutral fixation probability imply likely fixation?**

**Answer.** No. If neutral fixation probability is one percent, twice that baseline is two percent. The relative improvement is substantial while the absolute chance remains small.

**13. Why compare one initial copy with ten percent initial frequency?**

**Answer.** At one copy, increasing population size also lowers the initial frequency. At a fixed frequency, the initial count grows with population size. These are different questions and should not be mixed.

**14. Why should an approximation be checked against the exact result?**

**Answer.** An approximation simplifies an expression using assumptions about its parameters. Those assumptions can fail. Exact-versus-approximate error is a mathematical discrepancy, which more simulation replicates do not remove.

**15. What would you say about the contribution of this project?**

**Answer.** It derives and reproduces established results for a stated Moran model, validates a Julia implementation against an exact calculation, and investigates selection, drift, and approximation accuracy over a defined parameter range. Its purpose is to demonstrate understanding and careful scientific computation. It is not a claim of a new biological discovery.

## A.7 A final practice exercise with a full answer

**Task.** Explain the model to a friend in five sentences without using the terms Markov chain, backward equation, or Monte Carlo.

**One possible answer.** We follow two genetic types in a population whose size stays constant. At each step, one individual is chosen to reproduce and its offspring replaces another randomly selected individual. One type can have an advantage in the reproduction draw, but chance can still eliminate it while it is rare. We repeat the process many times and record how often that type eventually fills the population. We check the computer results against a probability calculated directly from the same rules.

Now give the technical version. Name the state, parameters, transition probabilities, absorbing boundaries, fixation-probability recurrence, and validation calculation. Being able to move between the ordinary and technical explanations is a stronger sign of understanding than being able to recite either version alone.

# Appendix B

# The complete runnable source

These listings use byte-for-byte copies of the runnable files in the supplied project. The preparation script refreshes them automatically before compilation. They are the same code explained in Chapters 5 and 10. The line numbers are for reading; they are not part of the programs. The numerical results and figures elsewhere in the guide were generated by running these files.

## B.1 The model: src/MoranModel.jl

```
module MoranModel

using Random
using LinearAlgebra

export validate_parameters, transition_probabilities, next_count
export simulate_trajectory, simulate_fixation, fixation_probability
export backward_probabilities, wilson_interval, deterministic_trajectory

# 1. Check that the numbers describe a valid population.
function validate_parameters(N, i, s)
    N isa Integer && N >= 2 || throw(ArgumentError("N must be an integer at least 2."))
    i isa Integer && 0 <= i <= N || throw(ArgumentError("i must be an integer from 0 to N."))
    s isa Real && isfinite(s) && s >= 0 && isfinite(Float64(s)) && (s == 0 || Float64(s) > 0)
     || throw(ArgumentError("s must be finite and nonnegative; a positive s must convert to a
     finite, positive Float64."))
    return nothing
end

# 2. Probabilities of gaining one copy, losing one copy, or staying put.
function transition_probabilities(N, i, s)
    validate_parameters(N, i, s)
    relative_fitness = 1.0 + Float64(s)
    # Dividing both reproductive weights by relative_fitness avoids overflow.
    weight_A = Float64(i)
    weight_B = (N - i) / relative_fitness
    total_weight = weight_A + weight_B
    parent_A = weight_A / total_weight
    parent_B = weight_B / total_weight
    up = parent_A * (N - i) / N
    down = parent_B * i / N
    stay = 1.0 - up - down
    return (up=up, down=down, stay=stay)
```

```
end

# 3. Turn a uniform random draw into the next allele count.
function next_count(i, up, down, u)
    0 <= u < 1 || throw(ArgumentError("u must be at least 0 and less than 1."))
    if u < up
        return i + 1
    elseif u < up + down
        return i - 1
    else
        return i
    end
end

# 4. Store one history. After absorption, repeat the same count to the horizon.
function simulate_trajectory(rng, N, i0, s, max_events)
    validate_parameters(N, i0, s)
    max_events isa Integer && max_events >= 0 || throw(ArgumentError("max_events must be a
    nonnegative integer."))
    counts = Vector{Int}(undef, max_events + 1)
    counts[1] = i0
    i = i0
    absorbed_at = nothing
    if i == 0 || i == N
        absorbed_at = 0
    end
    for event in 1:max_events
        if 0 < i < N
            probabilities = transition_probabilities(N, i, s)
            i = next_count(i, probabilities.up, probabilities.down, rand(rng))
            if i == 0 || i == N
                absorbed_at = event
            end
        end
        counts[event + 1] = i
    end
    return (counts=counts, absorbed=absorbed_at !== nothing, absorbed_at=absorbed_at)
end

# 5. Run until the allele is lost or fixed. There is no time cutoff.
function simulate_fixation(rng, N, i0, s)
    validate_parameters(N, i0, s)
    i = i0
    events = 0
    while 0 < i < N
        probabilities = transition_probabilities(N, i, s)
        i = next_count(i, probabilities.up, probabilities.down, rand(rng))
        events += 1
    end
    return (fixed=i == N, events=events)
end

# 6. Exact probability. log1p and expm1 preserve accuracy near neutrality.
function fixation_probability(N, i, s)
    validate_parameters(N, i, s)
    if i == 0
        return 0.0
    elseif i == N
        return 1.0
    elseif s == 0
```

```
        return i / N
    end
    log_fitness = log1p(Float64(s))
    numerator = -expm1(-i * log_fitness)
    denominator = -expm1(-N * log_fitness)
    return numerator / denominator
end

# 7. Independent check: solve the backward recurrence as linear equations.
function backward_probabilities(N, s)
    validate_parameters(N, 0, s)
    diagonal = zeros(N - 1)
    lower = zeros(N - 2)
    upper = zeros(N - 2)
    right_hand_side = zeros(N - 1)
    for i in 1:(N - 1)
        probabilities = transition_probabilities(N, i, s)
        diagonal[i] = probabilities.up + probabilities.down
        if i > 1
            lower[i - 1] = -probabilities.down
        end
        if i < N - 1
            upper[i] = -probabilities.up
        else
            right_hand_side[i] = probabilities.up
        end
    end
    matrix = Tridiagonal(lower, diagonal, upper)
    interior = matrix \ right_hand_side
    return vcat(0.0, interior, 1.0)
end

# 8. A 95% Wilson confidence interval for k successes in n repetitions.
function wilson_interval(k, n)
    n isa Integer && n > 0 || throw(ArgumentError("n must be a positive integer."))
    k isa Integer && 0 <= k <= n || throw(ArgumentError("k must be an integer from 0 to n."))
    z = 1.959963984540054
    estimate = k / n
    denominator = 1.0 + z^2 / n
    centre = (estimate + z^2 / (2n)) / denominator
    half_width = z * sqrt(estimate * (1 - estimate) / n + z^2 / (4n^2)) / denominator
    lower = max(0.0, centre - half_width)
    upper = min(1.0, centre + half_width)
    if k == 0
        lower = 0.0
    elseif k == n
        upper = 1.0
    end
    return (lower=lower, upper=upper)
end

# 9. A deterministic mean-field comparison, using one-event time steps.
function deterministic_trajectory(N, i0, s, max_events)
    validate_parameters(N, i0, s)
    max_events isa Integer && max_events >= 0 || throw(ArgumentError("max_events must be
    nonnegative."))
    frequencies = Vector{Float64}(undef, max_events + 1)
    x = i0 / N
    frequencies[1] = x
    for event in 1:max_events
```

```
        change = (s * x / (1.0 + s * x)) * ((1.0 - x) / N)
        x = x + change
        frequencies[event + 1] = x
    end
    return frequencies
end

end
```

## B.2 The demonstration: scripts/demo.jl

```
# Run from the project folder: julia --project=. scripts/demo.jl
include(joinpath(@__DIR__, "..", "src", "MoranModel.jl"))
using .MoranModel
using Random
using Printf

function main()
    N = 5
    i = 2
    s = 0.5
    probabilities = transition_probabilities(N, i, s)
    println("ONE EVENT: N=5, i=2, s=0.5")
    @printf("up = %.3f, down = %.3f, stay = %.3f\n", probabilities.up, probabilities.down,
    probabilities.stay)
    println("\nA HAND TRACE (specified draws, not a random sample)")
    draws = [0.12, 0.72, 0.41, 0.95, 0.10, 0.03]
    for (event, u) in enumerate(draws)
        probabilities = transition_probabilities(N, i, s)
        i = next_count(i, probabilities.up, probabilities.down, u)
        @printf("event %d: draw %.2f -> i = %d\n", event, u, i)
    end

    println("\nA SEEDED EXPERIMENT: N=100, i0=1, s=0.01")
    rng = MersenneTwister(20261027)
    repetitions = 2000
    fixations = 0
    for repetition in 1:repetitions
        outcome = simulate_fixation(rng, 100, 1, 0.01)
        fixations += outcome.fixed
    end
    interval = wilson_interval(fixations, repetitions)
    @printf("fixations: %d / %d\n", fixations, repetitions)
    @printf("estimate: %.6f\n", fixations / repetitions)
    @printf("95%% Wilson interval: [%.6f, %.6f]\n", interval.lower, interval.upper)
    @printf("exact probability: %.6f\n", fixation_probability(100, 1, 0.01))
end

main()
```

## B.3 The full study script

```
# Run from the project folder: julia --project=. scripts/run_study.jl
```

```julia
# For a faster check: julia --project=. scripts/run_study.jl --quick
include(joinpath(@__DIR__, "..", "src", "MoranModel.jl"))
using .MoranModel
using Random
using Printf
using Dates

# 1. A small helper writes one row of comma-separated values.
function write_row(file, values)
    println(file, join(values, ","))
end

# 2. Repeat the experiment for the fixed parameter grid.
function run_fixation_grid(output_directory, repetitions, base_seed)
    output_path = joinpath(output_directory, "fixation.csv")
    open(output_path, "w") do file
        println(file, "N,s,i0,repetitions,fixations,estimate,lower,upper,exact,standard_error,events_mean,seed")
        cell = 0
        for N in [20, 50, 100, 200]
            for s in [0.0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1]
                cell += 1
                seed = base_seed + cell
                rng = MersenneTwister(seed)
                fixations = 0
                total_events = 0
                for repetition in 1:repetitions
                    outcome = simulate_fixation(rng, N, 1, s)
                    fixations += outcome.fixed
                    total_events += outcome.events
                end
                estimate = fixations / repetitions
                interval = wilson_interval(fixations, repetitions)
                exact = fixation_probability(N, 1, s)
                standard_error = sqrt(estimate * (1 - estimate) / repetitions)
                write_row(file, (N, s, 1, repetitions, fixations, estimate,
                    interval.lower, interval.upper, exact, standard_error,
                    total_events / repetitions, seed))
                flush(file)
                @printf("cell %2d/28: N=%3d s=%.3f estimate=%.6f exact=%.6f\n", cell, N, s, estimate, exact)
            end
        end
    end
end

# 3. Compare one copy with an initial frequency of ten percent, analytically.
function write_initial_frequency_comparison(output_directory)
    open(joinpath(output_directory, "initial_frequency.csv"), "w") do file
        println(file, "N,s,i0,initial_frequency,exact_single,exact_ten_percent,relative_to_neutral_single")
        for N in [20, 50, 100, 200]
            for s in [0.0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1]
                i0 = div(N, 10)
                single = fixation_probability(N, 1, s)
                multiple = fixation_probability(N, i0, s)
                write_row(file, (N, s, i0, i0 / N, single, multiple, N * single))
            end
        end
    end
```

```
end

# 4. Measure the error in the weak-selection approximation.
function write_approximation_comparison(output_directory)
    open(joinpath(output_directory, "approximation.csv"), "w") do file
        println(file, "N,s,Ns,exact,approximation,relative_error")
        selection_values = 10.0 .^ range(log10(1e-5), log10(0.5), length=121)
        for N in [20, 50, 100, 200]
            for s in selection_values
                exact = fixation_probability(N, 1, s)
                approximation = s / (-expm1(-N * s))
                relative_error = abs(approximation - exact) / exact
                write_row(file, (N, s, N * s, exact, approximation, relative_error))
            end
        end
    end
end

# 5. Save eight preselected histories per condition for plotting.
function write_trajectories(output_directory, base_seed)
    N = 100
    max_events = 30000
    sample_every = 100
    paths_per_condition = 8
    condition = 0
    summary_path = joinpath(output_directory, "trajectory_summary.csv")
    open(summary_path, "w") do summary
        println(summary, "N,i0,s,path,seed,absorbed,absorbed_at,final_count")
        for i0 in [1, 10]
            for s in [0.0, 0.01, 0.1]
                condition += 1
                paths = Vector{Vector{Int}}()
                for path_number in 1:paths_per_condition
                    seed = base_seed + 100000 + 100condition + path_number
                    trajectory = simulate_trajectory(MersenneTwister(seed), N, i0, s,
    max_events)
                    push!(paths, trajectory.counts)
                    absorbed_at = trajectory.absorbed_at
                    if absorbed_at === nothing
                        absorbed_at = -1
                    end
                    write_row(summary, (N, i0, s, path_number, seed,
                        trajectory.absorbed, absorbed_at, trajectory.counts[end]))
                end
                deterministic = deterministic_trajectory(N, i0, s, max_events)
                selection_label = replace(string(s), "." => "p")
                filename = "trajectory_N$(N)_i$(i0)_s$(selection_label).csv"
                open(joinpath(output_directory, filename), "w") do file
                    println(file, "event,generation,path_1,path_2,path_3,path_4,path_5,path_6
    ,path_7,path_8,deterministic")
                    for event in 0:sample_every:max_events
                        row = Float64[event, event / N]
                        for counts in paths
                            push!(row, counts[event + 1] / N)
                        end
                        push!(row, deterministic[event + 1])
                        write_row(file, row)
                    end
                end
            end
```

```
        end
    end
end

# 6. Save the information needed to reproduce the run.
function write_metadata(output_directory, repetitions, base_seed, elapsed_seconds, quick)
    open(joinpath(output_directory, "metadata.txt"), "w") do file
        println(file, "Moran model simulation study")
        println(file, "Julia version: ", VERSION)
        println(file, "Random generator: Random.MersenneTwister")
        println(file, "Base seed: ", base_seed)
        println(file, "Cell seeds: base_seed + cell, in N-then-s loop order")
        println(file, "Trajectory seeds: base_seed + 100000 + 100*condition + path_number")
        println(file, "Population sizes: 20, 50, 100, 200")
        println(file, "Selection advantages: 0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1")
        println(file, "Initial copies for simulation study: 1")
        println(file, "Repetitions per cell: ", repetitions)
        println(file, "Quick demonstration mode: ", quick)
        println(file, "No external Julia packages are required.")
        println(file, "Every fixation experiment runs until loss or fixation.")
        println(file, "Trajectory display horizon: 30000 events; 100 events per plotted
    generation.")
        println(file, "trajectory_summary absorbed_at=-1 means unfinished at the display
    horizon.")
        println(file, "Elapsed seconds: ", elapsed_seconds)
        println(file, "Completed UTC: ", Dates.now(Dates.UTC))
    end
end

# 7. Choose a run mode, then generate all the data.
function main()
    quick = "--quick" in ARGS
    unknown = setdiff(ARGS, ["--quick"])
    isempty(unknown) || error("Unknown arguments: $(join(unknown, ", "))")
    repetitions = 20000
    mode = "full"
    if quick
        repetitions = 1000
        mode = "demo"
    end
    base_seed = 20261027
    output_directory = normpath(joinpath(@__DIR__, "..", "results", mode))
    mkpath(output_directory)
    start_time = time()
    run_fixation_grid(output_directory, repetitions, base_seed)
    write_initial_frequency_comparison(output_directory)
    write_approximation_comparison(output_directory)
    write_trajectories(output_directory, base_seed)
    elapsed_seconds = time() - start_time
    write_metadata(output_directory, repetitions, base_seed, elapsed_seconds, quick)
    @printf("Finished in %.2f seconds. Data: %s\n", elapsed_seconds, output_directory)
end

main()
```

## B.4 The automated checks: test/runtests.jl

The Test library marks expressions that must be true. `@test` checks an ordinary assertion. `@test_throws` checks that an invalid request raises the expected error rather than returning a misleading number. `@testset` groups related checks. The function `isapprox` tests numerical agreement within tolerance; exact equality would be inappropriate for independently rounded floating-point calculations. The Monte Carlo check has a deliberately generous tolerance measured in standard errors; it does not demand that all nominal 95% confidence intervals cover.

```
include(joinpath(@__DIR__, "..", "src", "MoranModel.jl"))
using .MoranModel
using Random
using Test

@testset "Valid parameters and event rules" begin
    @test_throws ArgumentError transition_probabilities(1, 0, 0.0)
    @test_throws ArgumentError transition_probabilities(5, 6, 0.0)
    @test_throws ArgumentError transition_probabilities(5, 1.5, 0.0)
    @test_throws ArgumentError transition_probabilities(5, 1, -0.01)
    @test_throws ArgumentError transition_probabilities(5, 1, Inf)
    @test_throws ArgumentError transition_probabilities(5, 1, big"1e-500")
    @test_throws ArgumentError transition_probabilities(5, 1, big"1e500")
    @test_throws ArgumentError next_count(2, 0.3, 0.2, 1.0)
    example = transition_probabilities(5, 2, 0.5)
    @test isapprox(example.up, 0.3)
    @test isapprox(example.down, 0.2)
    @test isapprox(example.stay, 0.5)
    @test next_count(2, 0.3, 0.2, 0.12) == 3
    @test next_count(2, 0.3, 0.2, 0.41) == 1
    @test next_count(2, 0.3, 0.2, 0.72) == 2
    for N in [2, 5, 20], i in 0:N, s in [0.0, 1e-12, 0.01, 0.5, 1e300]
        p = transition_probabilities(N, i, s)
        @test all(isfinite, values(p))
        @test minimum(values(p)) >= -1e-14
        @test isapprox(sum(values(p)), 1.0)
    end
end

@testset "Exact probabilities and independent recurrence" begin
    for N in [2, 5, 20, 100], s in [0.0, 1e-12, 0.01, 0.1, 0.5]
        numerical = backward_probabilities(N, s)
        for i in 0:N
            exact = fixation_probability(N, i, s)
            @test 0.0 <= exact <= 1.0
            @test isapprox(exact, numerical[i + 1]; atol=1e-11, rtol=1e-10)
            if s == 0
                @test exact == i / N
            end
        end
    end
    @test isapprox(fixation_probability(5, 2, 0.5), 135 / 211)
    @test isapprox(fixation_probability(100, 1, 1e-16), 0.01)
    @test fixation_probability(100, 1, 0.1) > fixation_probability(100, 1, 0.01)
    @test fixation_probability(200, 1, 0.1) < fixation_probability(20, 1, 0.1)
end

@testset "Histories, boundaries and deterministic drift" begin
    lost = simulate_trajectory(MersenneTwister(1), 5, 0, 0.1, 10)
```

```
    fixed = simulate_trajectory(MersenneTwister(1), 5, 5, 0.1, 10)
    @test lost.counts == zeros(Int, 11)
    @test fixed.counts == fill(5, 11)
    @test lost.absorbed && lost.absorbed_at == 0
    @test simulate_fixation(MersenneTwister(1), 5, 0, 0.1) == (fixed=false, events=0)
    @test simulate_fixation(MersenneTwister(1), 5, 5, 0.1) == (fixed=true, events=0)
    path1 = simulate_trajectory(MersenneTwister(123), 20, 2, 0.1, 1000)
    path2 = simulate_trajectory(MersenneTwister(123), 20, 2, 0.1, 1000)
    @test path1 == path2
    @test all(0 .<= path1.counts .<= 20)
    @test all(abs.(diff(path1.counts)) .<= 1)
    @test deterministic_trajectory(5, 2, 0.0, 20) == fill(0.4, 21)
    @test isapprox(deterministic_trajectory(5, 2, 0.5, 1)[2], 0.42)
    @test all(diff(deterministic_trajectory(20, 2, 0.1, 1000)) .>= 0)
end

@testset "Confidence intervals and Monte Carlo sanity check" begin
    @test wilson_interval(0, 100).lower == 0.0
    @test wilson_interval(100, 100).upper == 1.0
    @test isapprox(wilson_interval(50, 100).lower, 0.4038315303659956)
    @test_throws ArgumentError wilson_interval(1, 0)
    rng = MersenneTwister(817)
    repetitions = 10000
    fixations = 0
    for repetition in 1:repetitions
        fixations += simulate_fixation(rng, 20, 1, 0.1).fixed
    end
    exact = fixation_probability(20, 1, 0.1)
    sampling_error = sqrt(exact * (1 - exact) / repetitions)
    @test abs(fixations / repetitions - exact) <= 6sampling_error + 1 / repetitions
end
```

## B.5 Preparing the guide's plot inputs

This helper does not add new biological rules or new stochastic experiments. It reads the completed full study, separates rows by population size, calculates error-bar lengths, and writes small data files that LaTeX can plot. It also prepares the printed tables and summary counts. `readlines` reads a file as text lines; `split` separates a CSV row at commas; `parse` converts a text field to a number. The dot in `String.(...)` applies conversion to each field. `push!` appends a row to an array.

The first preparation block also copies the runnable source and trajectory data into the guide folder. `cp` copies a file, and `force=true` allows replacing an older generated copy. `basename` keeps just the filename. `readdir` lists the data directory; `startswith` and `endswith` identify the trajectory CSV files. These copies keep the LaTeX project self-contained without asking its compiler to read outside its folder. Edit the original runnable code, then run this helper again; do not edit the generated copies separately.

`mkpath` creates the output directory if needed. `open(..., "w") do file` writes through the file handle and closes it at the end. The helper checks for 28 conditions and 20,000 repetitions per condition so that a quick run is not accidentally used as the full-study report. An inequality added to `covered` contributes one for true and zero for false. The standardised discrepancy divides the estimate's error by the exact binomial sampling standard error. This differs from

the plug-in standard error stored in the study CSV, which uses the observed proportion.

The first loop writes summary numbers. The next writes the selected $N = 100$ table. For each population size, later loops prepare estimated points, a smooth exact curve, and the approximation-error data. The final loop prepares the initial-frequency comparison. The `@printf` instructions format LaTeX table rows; doubled backslashes in the string represent literal backslashes required by LaTeX. They do not alter a scientific calculation.

```
# Run after the full study: julia --project=. scripts/prepare_guide.jl
# This only prepares tables and plot inputs. It does not run new experiments.
include(joinpath(@__DIR__, "..", "src", "MoranModel.jl"))
using .MoranModel
using Printf

function read_rows(path)
    lines = readlines(path)
    rows = Vector{Vector{String}}()
    for line in lines[2:end]
        push!(rows, String.(split(line, ",")))
    end
    return rows
end

function main()
    root = normpath(joinpath(@__DIR__, ".."))
    data = joinpath(root, "results", "full")
    target = joinpath(root, "guide", "generated")
    mkpath(target)
    # Keep the LaTeX project self-contained for the app and for Overleaf.
    code_directory = joinpath(root, "guide", "code")
    mkpath(code_directory)
    for source in ["src/MoranModel.jl", "scripts/demo.jl", "scripts/run_study.jl",
                   "scripts/prepare_guide.jl", "test/runtests.jl"]
        cp(joinpath(root, source), joinpath(code_directory, basename(source)); force=true)
    end
    for filename in readdir(data)
        if startswith(filename, "trajectory_N") && endswith(filename, ".csv")
            cp(joinpath(data, filename), joinpath(target, filename); force=true)
        end
    end
    rows = read_rows(joinpath(data, "fixation.csv"))
    length(rows) == 28 || error("Run the complete study first: expected 28 conditions.")
    covered = 0
    total_repetitions = 0
    maximum_standardised_error = 0.0
    for row in rows
        repetitions = parse(Int, row[4])
        repetitions == 20000 || error("The teaching guide requires the full 20,000-repetition study.")
        total_repetitions += repetitions
        estimate = parse(Float64, row[6])
        exact = parse(Float64, row[9])
        covered += parse(Float64, row[7]) <= exact <= parse(Float64, row[8])
        error_in_units_of_se = abs(estimate - exact) / sqrt(exact * (1 - exact) / repetitions)
        maximum_standardised_error = max(maximum_standardised_error, error_in_units_of_se)
    end
    open(joinpath(target, "summary.tex"), "w") do file
        println(file, "\\newcommand{\\StudyRepetitions}{", total_repetitions, "}")
        println(file, "\\newcommand{\\IntervalsCovering}{", covered, "}")
        @printf(file, "\\newcommand{\\MaximumStandardisedError}{%.3f}\n",
    maximum_standardised_error)
    end
    open(joinpath(target, "selected_table.tex"), "w") do file
```

```
        println(file, raw"\begin{tabular}{rrrrl}\toprule")
        println(file, raw"\(s\) & Fixations & Estimate & Exact & 95\% Wilson interval\\\midrule")
        for row in rows
            if parse(Int, row[1]) == 100
                @printf(file, "%.3f & %s & %.4f & %.4f & [%.4f, %.4f] \\\\\n",
                    parse(Float64, row[2]), row[5], parse(Float64, row[6]),
                    parse(Float64, row[9]), parse(Float64, row[7]), parse(Float64, row[8]))
            end
        end
        println(file, raw"\bottomrule\end{tabular}")
    end
    for N in [20, 50, 100, 200]
        open(joinpath(target, "fixation_N$(N).csv"), "w") do file
            println(file, "s,estimate,exact,error_lower,error_upper")
            for row in rows
                if parse(Int, row[1]) == N
                    estimate = parse(Float64, row[6])
                    println(file, join((row[2], estimate, row[9],
                        estimate - parse(Float64, row[7]), parse(Float64, row[8]) - estimate), ",")
)
                end
            end
        end
        open(joinpath(target, "curve_N$(N).csv"), "w") do file
            println(file, "s,exact")
            for s in range(0.0, 0.1, length=201)
                println(file, s, ",", fixation_probability(N, 1, s))
            end
        end
        approximation_lines = readlines(joinpath(data, "approximation.csv"))
        open(joinpath(target, "approximation_N$(N).csv"), "w") do file
            println(file, approximation_lines[1])
            for line in approximation_lines[2:end]
                if first(split(line, ",")) == string(N)
                    println(file, line)
                end
            end
        end
    end
    open(joinpath(target, "initial_table.tex"), "w") do file
        println(file, raw"\begin{tabular}{rrrr}\toprule")
        println(file, raw"\(N\) & One copy & Initial frequency 10\% & One copy / neutral\\\midrule
")
        for row in read_rows(joinpath(data, "initial_frequency.csv"))
            if parse(Float64, row[2]) == 0.01
                @printf(file, "%s & %.4f & %.4f & %.3f \\\\\n", row[1],
                    parse(Float64, row[5]), parse(Float64, row[6]), parse(Float64, row[7]))
            end
        end
        println(file, raw"\bottomrule\end{tabular}")
    end
    println("Prepared guide inputs from ", total_repetitions, " completed repetitions.")
end

main()
```

# Bibliography

[1] D. M. McCandlish, C. L. Epstein, and J. B. Plotkin. Formal properties of the probability of fixation: identities, inequalities and approximations. *Theoretical Population Biology*, 99:98–113, 2015. doi:10.1016/j.tpb.2014.11.004. Open manuscript: https://arxiv.org/abs/1312.1673.

[2] The Julia Project. *Julia documentation: getting started, functions, control flow, and random numbers*. https://docs.julialang.org/en/v1/. Consulted 27 October 2026.

[3] NIST/SEMATECH. *e-Handbook of Statistical Methods*, section 7.2.4.1: confidence intervals for a proportion. https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm. Consulted 27 October 2026.